



DATABASE SYSTEMS: DESIGN AND IMPLICATION

CST2340- COURSEWORK

Name:

Reeva Josephine Deepak: M01036971

Mutsawashe Audrey Mandinde M01033270

Table of Contents

COVER PAGE	1
Part A – Database Design.....	3
A1. Case Study Description.....	3
A2. Entity-Relationship Model.....	4
A3. Model Validation.....	6
A4. Limitations of the Relational Model and NoSQL Comparison	9
A5. Professional, Legal and Ethical Issues in Database Design and Management	17
Part B – Database Implementation	24
B1. Relational Model Diagram	24
B2. Convert Relational Model into Tables.....	24
B3. Tables Creation.....	26
Part C – SQL Codes	50
CONTRIBUTION TABLE:	69
REFERENCES LIST:	72

Part A – Database Design

A1. Case Study Description

CASE STUDY:

Totally Toothy Dental Clinic is a dental healthcare provider that offers a variety of dental treatments to patients. Each treatment is identified by a unique TreatmentID and includes details such as the treatment name, description, price, duration, and number of sessions required. Treatments offered by the clinic include teeth cleaning, tooth fillings, extractions, whitening, braces, and cosmetic procedures such as tooth gem installations.

The clinic employs qualified doctors, including general dentists, orthodontists, hygienists, and oral surgeons. Each doctor is identified by a unique DoctorID and includes information such as full name, contact number, email address, role, and area of specialization. Doctors are responsible for diagnosing patients, performing treatments, and overseeing patient care during appointments.

Patients who attend the clinic are registered as patients, each identified by a unique PatientID. Patient records include full name, contact number, email address, and home address. A patient may have one or more drug allergies, which are stored separately using an AllergyID along with the allergy name and description. A patient can be associated with multiple allergies, and the same allergy may apply to multiple patients.

Patients receive treatments by booking appointments. Each appointment is identified by an AppointmentID and includes the appointment date, appointment time, booking method (online or walk-in), and payment type. Each appointment is associated with one patient, one doctor, and one treatment.

Appointments take place in designated rooms within the clinic. Each room is identified by a RoomID and includes a room name and room category, such as surgery rooms or general treatment rooms. Each appointment is assigned one room to ensure that treatments are carried out in suitable facilities and to avoid scheduling conflicts.

Some appointments require additional clinical support from a dental assistant. Each dental assistant is identified by an AssistantID and includes basic contact information. An

appointment may involve a dental assistant depending on the type of treatment being performed.

For billing purposes, the clinic generates an invoice for each appointment. Each invoice is identified by an InvoiceID and includes the invoice date, total amount, payment method, and payment status. Each invoice is linked directly to one appointment.

For treatments that require multiple visits over an extended period of time, such as braces, the clinic creates a treatment plan. Each treatment plan is identified by a TreatmentPlanID and includes a start date, expected end date, and overall notes. A treatment plan is associated with one patient, one doctor (typically an orthodontist), and one treatment. Multiple appointments may be scheduled under a treatment plan to allow continuous monitoring, adjustments, and progress tracking throughout the course of the treatment.

A2. Entity-Relationship Model

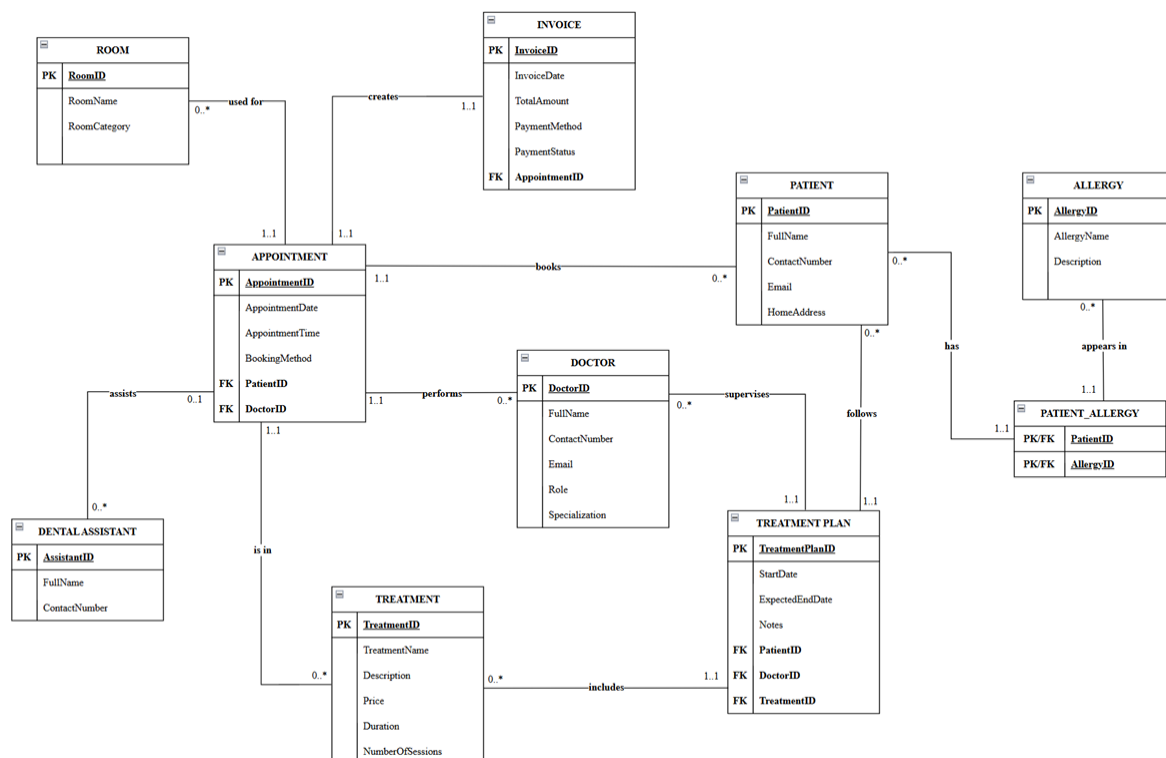


Figure 1 Totally Toothy Dental Clinic ERD

ERD RELATIONSHIPS:

1. Patient -- Appointment

A Patient books many Appointments, and each Appointment is booked by one Patient.

2. Doctor -- Appointment

A Doctor performs many Appointments, and each Appointment is performed by one Doctor.

3. Treatment -- Appointment

A Treatment is in many Appointments, and each Appointment includes one Treatment.

4. Room -- Appointment

A Room is used for many Appointments, and each Appointment takes place in one Room.

5. Dental Assistant -- Appointment

A Dental Assistant assists in many Appointments, and an Appointment may be assisted by one or more Dental Assistants.

6. Appointment -- Invoice

An Appointment creates one Invoice, and each Invoice is created from one Appointment.

7. Patient -- Patient_Allergy -- Allergy

A Patient has many Patient_Allergy records, and each Patient_Allergy applies to one Allergy.

An Allergy applies to many Patient_Allergy records, and each Patient_Allergy belongs to one Patient.

8. Patient -- Treatment Plan

A Patient follows many Treatment Plans, and each Treatment Plan is followed by one Patient.

9. Doctor -- Treatment Plan

A Doctor supervises many Treatment Plans, and each Treatment Plan is supervised by one Doctor.

10. Treatment -- Treatment Plan

A Treatment includes many Treatment Plans, and each Treatment Plan is for one Treatment.

A3. Model Validation

Identifying the Connection Trap

When reviewing the ERD, a possible chasm trap was found between the TREATMENT PLAN and APPOINTMENT entities. A treatment plan is linked to both a patient and a doctor, and an appointment is also linked to a patient and a doctor, but there's no direct relationship between treatment plan and appointment.

This becomes a problem when a patient has several treatment plans. For example, if a patient has one plan for braces and another for regular cleaning, the system would not be able to identify which appointments belong to which plan. The model would only show that the appointments are for that patient, without specifying the related treatment plan.

A chasm trap happens when two entities look related through other entities, but there's no real connection that allows proper navigation between them. Here, even though TREATMENT PLAN and APPOINTMENT both connect to PATIENT and DOCTOR, the missing direct link leaves a gap in the model.

Resolution of the Trap

To fix this issue, a foreign key called TreatmentPlanID was added to the APPOINTMENT entity. The new relationship is:

- One Treatment Plan can have many Appointments.
- Each Appointment may optionally belong to one Treatment Plan.

This update allows appointments under long-term treatments (like braces) to be linked to the correct treatment plan. It eliminates ambiguity and helps keep the data more reliable.

Access Path

TREATMENT PLAN → APPOINTMENT → INVOICE

Explanation

The TreatmentPlanID is used to find all the appointments linked to a specific braces treatment plan. A condition can then be added to the AppointmentDate to only show sessions booked after a certain date, such as 01/03/2026.

From the APPOINTMENT entity, the system can access the related INVOICE records to check the PaymentStatus for each appointment. This setup allows the clinic to keep track of both the medical progress of the treatment and the financial side of it at the same time.

Overall, this shows that the updated ERD now supports the clinic's needs for both daily operations and billing management in long-term treatments like braces.

A4. Limitations of the Relational Model and NoSQL Comparison

Introduction

The relational model has been the dominant approach to database design for decades and remains highly effective for many applications. However, as data volumes grow and organisations need to handle increasingly diverse types of information, certain limitations of this model have become more apparent. Understanding these constraints helps explain why alternative models like NoSQL have emerged and why a clinic like Totally Toothy might eventually need to look beyond traditional relational databases (Tsiakara, 2025b).

Advantages of the Relational Model

The structure is simple and intuitive, with data organised into tables of rows and columns that most users find easy to understand. Querying is powerful because users can combine related tables and filter results based on various criteria. Data integrity is enforced through primary and foreign keys, which prevent incomplete or orphaned records. Normalisation helps eliminate redundancy and ensures the database design is robust (Tsiakara, 2025b).

For Totally Toothy, these strengths are important. The relationships between patients, appointments, doctors and treatments fit naturally into a relational model structure. Referential integrity ensures every appointment links to a valid patient and doctor and also normalisation prevents duplicate data. This shows us that the relational model is a good fit for what a dental clinic needs.

Limitations of the Relational Model

Despite these advantages, the relational model has limitations that become problematic in a lot of ways.

Rigidity of Structure

The relational model requires all data to conform to a fixed tabular format. Every row in a table must have the same columns, regardless of whether those columns are appropriate for every record. This rigidity means the schema must be designed in advance, and modifying it later can be difficult (Tsiakara, 2025b).

For a dental clinic, this is mostly manageable because patient data is fairly predictable. However, if the clinic began offering new treatments requiring different types of information,

adding columns to the Treatment table would affect every row, even if the new fields only apply to a few treatments. The structure does not handle variation well.

Wasted Space from Optional Fields

When a table includes columns needed for only some records, the remaining records store null values, wasting storage space and complicating queries.

In the Totally Toothy database, the Patient_Allergy table avoids this by using a separate table for allergies rather than storing them in the Patient table. However if the clinic later wanted to store optional information like patient preferences or emergency contacts, they would face the choice of adding nullable columns to Patient or creating additional separate tables.

Difficulty Adding New Fields

Adding a new field to a relational table requires altering the table structure, which can lock the database and interrupt service. The change affects every row, even if the new field will be NULL for most existing records (Tsiakara, 2025b).

If Totally Toothy decided to start collecting patient social media handles for marketing, they would need to alter the Patient table. During that alteration, the database might become unavailable, and every existing patient record would gain a NULL value in the new column.

Poor Handling of Unstructured Data

Relational databases are designed for structured data that fits neatly into rows and columns. They struggle with unstructured data such as images, videos and long text documents.

A dental clinic generates X-rays, photographs of patients' teeth, and potentially video consultations. Storing these in a relational database is possible using BLOB fields, but performance suffers and queries become complicated. The database is not optimised for this type of data.

Scalability Constraints

Relational databases scale vertically, meaning handling increased load requires upgrading to a more powerful server with faster processors, more memory and larger storage. There is a practical limit to how much a single server can handle, and high-end hardware becomes expensive quickly (IBM, 2022).

If Totally Toothy expands to multiple locations and needs to centralise patient records, a single server might eventually struggle. The only option is to continue investing in bigger hardware, which is not sustainable indefinitely.

Cost

Setting up and maintaining a relational database requires specialised software, trained staff and ongoing security measures. These costs accumulate, particularly for smaller organisations .IBM (2022) notes that commercial RDBMS can be expensive to maintain, requiring licenses, trained database managers, and powerful hardware for vertical scaling.

What is NoSQL

NoSQL stands for Not Only SQL. It is an approach to database design that enables storage and querying of data outside the traditional structures found in relational databases (IBM, 2022). Rather than storing data in tables with fixed schemas, NoSQL databases use more flexible structures that can adapt to the data being stored

The term does not imply these databases reject SQL entirely but it means they are not constrained by the relational approach and can handle data in ways that relational databases cannot. IBM (2022) explains that NoSQL databases house data within a single data structure, such as a JSON document, and since this design does not require a schema, it offers rapid scalability to manage large and typically unstructured data sets.

Types of NoSQL Databases

There are four main types of NoSQL databases, each suited to different kinds of problems.

Document Stores

Document databases store data as documents, typically in formats like JSON or XML. Each document can have different fields, and the structure can vary from one document to another. This flexibility makes them suitable for situations where data is not perfectly uniform. IBM (2022) notes that document stores are helpful for managing semi-structured data and are commonly used for content management systems and user profiles.

For a dental clinic, a document store could hold each patient record as a single document containing personal details, allergies, appointment history and treatment notes all in one place. This avoids the joins that relational databases require.

Key-Value Stores

Key-value stores are the simplest NoSQL databases. They function like a dictionary or map, where each piece of data is stored with a unique key. Retrieving data means looking up the key and obtaining the associated value. IBM (2022) describes this schema-less data model as organised into a dictionary of key-value pairs, commonly used for caching and storing user session information such as shopping carts.

These are useful for caching or storing session data, but they are not ideal for complex relationships.

Column Family Stores

Column family databases store information in columns rather than rows, enabling users to access only the specific columns they need without allocating additional memory on irrelevant data (IBM, 2022). While this may sound similar to relational databases, the key difference is that each row can have different columns. They are designed for handling massive amounts of data across many servers.

A clinic analysing treatment outcomes over years might use a column store for research data while keeping patient records in a relational system.

Graph Databases

Graph databases focus on relationships between data points. They store nodes representing entities and edges representing connections between them. IBM (2022) explains that data elements are stored as nodes, edges and properties, with edges defining the relationship between nodes. This makes them extremely efficient for queries that involve navigating relationships.

If Totally Toothy wanted to understand how patients are connected through family referrals or track how treatments lead to follow-up appointments, a graph database would excel at this.

How NoSQL Addresses Relational Limitations

Flexible Schema

Where relational databases require a fixed schema defined in advance, NoSQL databases allow different records to have different fields. Since NoSQL database design does not require a schema, it offers rapid scalability to manage large and typically unstructured data

sets (IBM, 2022). This addresses the rigidity problem and makes it easier to evolve the data model over time.

If the clinic wanted to start storing social media handles for marketing, they could simply add that field to new patient documents without altering any existing records.

No Wasted Space

Because NoSQL documents only include the fields that actually exist for each record, there is no wasted space storing NULL values. This is more efficient for sparse data.

Handling Unstructured Data

NoSQL databases are designed to handle unstructured and semi-structured data natively.

Storing X-ray images alongside patient records is more natural in a document store than in a relational database.

Horizontal Scaling

Relational databases can only scale vertically, which limits your ability to add more memory to the hardware you have (IBM, 2022). NoSQL databases scale horizontally. Their built-in sharding and high availability capabilities ease horizontal scaling, meaning if a single database server is not enough to store all your data or handle all the queries, the workload can be divided across two or more servers (IBM, 2022). This approach is more cost-effective and has no theoretical limits.

A multi-location clinic could scale by adding database servers as new branches open, rather than purchasing one massive central server.

Replication and Availability

NoSQL replication functionality copies and stores data across multiple servers, providing data reliability, ensuring access during down time, and protecting against data loss if servers go offline (IBM, 2022).

Speed

NoSQL enables faster, more agile storage and processing, making it generally a better fit for modern, complex web applications (IBM, 2022).

Advantages of NoSQL

IBM (2022) summarises the key advantages of NoSQL databases as:

- **Cost-effectiveness:** NoSQL databases allow you to quickly scale horizontally, better allocating resources to minimise costs, compared to expensive high-end commercial RDBMS.
- **Flexibility:** Horizontal scaling and a flexible data model mean NoSQL databases can address large volumes of rapidly changing data, making them great for agile development.
- **Replication:** Data is copied and stored across multiple servers, ensuring access during downtime and protecting against data loss.
- **Speed:** NoSQL enables faster, more agile storage and processing for all users.

Application to Totally Toothy Dental Clinic

For a dental clinic with predictable data and clear relationships, the relational model is a perfectly appropriate choice. The current database design, with its separate tables for patients, appointments, treatments and invoices, fits naturally into a relational system.

However, as the clinic grows and its needs become more complex, there may be situations where NoSQL becomes relevant. X-ray images and photographs could be stored more efficiently in a document store alongside patient records. Analysing treatment patterns over time might benefit from a column store designed for analytics. Understanding patient referral networks could be easier with a graph database.

This does not mean abandoning the relational database. IBM (2022) notes that companies commonly utilise both NoSQL and relational databases in a single application, an approach known as polyglot persistence. The relational system would remain the source of truth for appointments and billing, while other databases handle specific tasks.

Other Database Models

Beyond NoSQL, several other alternative models are worth mentioning briefly. In-memory databases store data in main memory rather than on disk, making them extremely fast. Streaming databases process data in real time as it arrives. Columnar databases store data by column rather than row, making them efficient for analytical queries. Virtualised databases provide a single interface to query multiple underlying databases without moving or copying data (Tsiakara, 2025b).

Conclusion

The relational model has served database design well for decades and remains the right choice for many applications, including Totally Toothy Dental Clinic. Its strengths in data integrity, structured querying and relationship management align with the needs of a healthcare setting.

However, the model has genuine limitations. Rigid schemas, poor handling of unstructured data and vertical scaling constraints create challenges as data grows and diversifies. NoSQL databases address these limitations through flexible schemas, native support for unstructured data and horizontal scaling (IBM, 2022).

For the clinic, understanding these alternatives is valuable even if the current implementation remains relational. As technology evolves and patient data becomes more complex, having options beyond the traditional model ensures the clinic can adapt without being locked into a single approach.

But the model has real limitations. Rigid schemas, poor handling of unstructured data and vertical scaling constraints create problems as data grows and diversifies. NoSQL databases address these limitations through flexible schemas, native support for unstructured data and horizontal scaling.

For the clinic, understanding these alternatives is valuable even if the current implementation stays relational. As technology evolves and patient data becomes more complex, having options beyond the traditional model ensures the clinic can adapt without being locked into a single approach.

A5. Professional, Legal and Ethical Issues in Database Design and Management

When designing and managing a database, organisations need to think about a lot of professional, legal, and ethical issues. Databases usually hold a huge amount of sensitive information such as personal data, financial details, or company records. If this kind of information gets leaked, misused, or accessed by people who shouldn't have it, it can cause serious harm to both individuals and organisations. Because of that, database designers and administrators have to make sure they follow proper laws, professional standards, and ethical rules to keep data safe and handled responsibly.

Data Privacy

One of the biggest and most important issues in database management is data privacy. Databases usually contain a lot of personal info, like people's names, contact details, ID numbers, and even financial information. Keeping this private is really important because if it's exposed or used wrongly, it could lead to identity theft, money loss, or even damage someone's reputation.

Research also shows that keeping privacy protected is one of the main responsibilities in database management, because every person has the right to have their information treated with respect (Pina et al., 2024). A big principle that's often used is called data minimisation, which means that organisations should only collect and store the data they actually need. Keeping extra data that isn't necessary just increases the chances of it being misused or stolen (Pina et al., 2024).

Another important concept is privacy by design, which means protecting privacy should be part of the system from the very start, instead of being added at the end (Cavoukian, 2009). When systems are built with privacy already in mind, it becomes much less likely that personal data will be exposed or misused.

Data Security

Data security is another major issue in database management. Databases must be protected from hackers, cyberattacks, and also from data getting lost by accident. If a database isn't set up properly, it can easily be attacked through things like SQL injection, where a hacker types in harmful commands so they can break in and see data they shouldn't (The Computing Teacher, 2024).

To stop something like that, database administrators need to use proper security methods such as:

- Encrypting sensitive data
- Using strong user authentication systems
- Setting up clear access control rules
- Doing regular backups and system checks

Security is also strongly connected to privacy and trust. If a system fails to keep people's data safe, it not only breaks privacy but also makes people lose trust in the organisation (Lee, Zankl and Chang, 2016). That's why database professionals have a huge responsibility to make sure all the right security measures are not just applied but also updated regularly.

Legal Issues and Data Protection Laws

Database management also has to follow the law. There are data protection laws that say how organisations can collect, store, and use personal data, and these laws exist to make sure people's information isn't misused.

A good example of this is the General Data Protection Regulation (GDPR). It requires organisations to follow strict rules about how they handle personal data. For example, companies need to get permission from people before collecting their information, and they must also let people see or delete their own data if they want.

There are also laws about who owns and controls the data that's stored in a system. Data governance is really important because it makes sure sensitive information is managed responsibly and legally (International Society of Nephrology, 2024). If an organisation fails to follow these laws, it can lead to big fines, legal trouble, and serious damage to its reputation.

Ethical Use of Data

Besides legal rules, organisations should also care about using data ethically. Ethical use of data means handling it in a way that's respectful, fair, and responsible.

One major ethical rule is called informed consent. This means that people should know how their data is going to be used and must agree to it before it gets collected. Another important

principle is fairness, which means data should never be used to discriminate or treat people unfairly (Florea and Florea, 2020).

Ethical guidelines in technology focus on things like respect for individuals, justice, and reducing harm when dealing with people's data (Menlo Report, 2012). For example, organisations should make sure they don't use someone's personal information for things like unfair profiling or making biased decisions.

Data Accuracy and Integrity

Keeping data accurate and maintaining its integrity is another important responsibility in database management. A database needs to store correct and reliable information because if the data is wrong, it can cause serious problems.

For example, wrong details in a system could lead to mistakes in banking, healthcare, or business decisions. Data integrity means keeping data correct, consistent, and trustworthy throughout its whole lifecycle.

Research has shown that poor data management can lead to bad results like financial losses, operational problems, and damage to a company's reputation (Faridoon and Kechadi, 2024). To prevent this, databases should have validation rules, be updated often, and have audit logs that record all the changes made to the data so issues can be found and fixed easily.

Data Ownership and Access Rights

Another really important issue in database management is figuring out who owns the data and who's allowed to access it. Databases can have information from users, companies, or even outside sources, so it's important to make sure there's a clear rule about who has control over it.

Usually, individuals have certain rights over their own personal data. They can ask to see it, update it, or even have it deleted in some cases. This is known as the right to be forgotten (Pina et al., 2024).

Access to data should also be limited depending on a person's role in the organisation. A system called role-based access control helps with this, as it allows people to only access the data they actually need for their work. Handling access rights properly is one of the best ways to stop data misuse and keep sensitive information safe.

Conclusion

In conclusion, designing and managing databases isn't just about storing information. It also involves professional, legal, and ethical responsibilities. Organisations need to make sure they protect personal data, use secure systems, and follow data protection laws. They also have to stay fair, honest, and respectful when it comes to handling people's data.

By paying attention to data privacy, data security, legal rules, ethical use, accuracy, and access rights, organisations can protect sensitive information and build trust with their users. As technology keeps developing, managing databases properly is going to be even more important to stop data misuse, protect people's privacy, and keep information safe.

Application of Professional, Legal and Ethical Issues to the Totally Toothy Dental Clinic Database

The professional, legal, and ethical principles mentioned earlier are really important when creating and managing the database for Totally Toothy Dental Clinic. The clinic's database stores lots of sensitive healthcare information like patient details, allergy records, treatment history, appointment schedules, and billing data. Since all of this info is private and connected to people's health, the system needs to make sure it's managed carefully, safely, and in a way that follows legal rules.

Patient Privacy and Confidentiality

One of the most important things for the Totally Toothy database is keeping patient information private. The system stores details such as patient names, phone numbers, email and home addresses, plus information about drug allergies. This kind of data is extremely sensitive and can't be seen or used by people who aren't allowed. Only authorised staff should have access, and even then, what they can see should depend on their job role.

For example, dentists might need full access to allergy and treatment details so they can plan proper care, but admin staff would only need to see appointment times and billing info. Using something like role-based access control makes sure everyone only sees what's needed for their job. Keeping patient information private is a key part of healthcare data management.

Data Security in the Clinic System

The Totally Toothy database also has to be very secure to stop anyone from getting in without permission, to protect it from hackers, or even from losing data by accident. The system includes data about patients, doctors, treatments, appointments, rooms, invoices, and treatment plans, so if someone got access to that, they could see or change private medical information, or even mess with the clinic's financial records.

To stop things like that, the clinic should use strong security steps like login systems with passwords, encrypted storage for sensitive data, and backing up data regularly. It's also important to watch database activity so suspicious actions can be spotted early. When building the system, safe coding methods should be used to prevent attacks like SQL injections, which can let hackers sneak into protected data. Having proper security in place keeps healthcare systems safe from these kinds of threats (Lee, Zankl and Chang, 2016).

Legal Compliance and Data Protection

Healthcare organisations always have to follow data protection laws when storing and managing people's personal details. When a new patient signs up at Totally Toothy Dental Clinic, their details are collected and stored in the system, but this should only be done for reasons linked to treatments or setting up appointments.

Patients also have the right to know how their data is used and stored. Under modern data protection laws, like the GDPR, people can ask to see their personal info and even request to change or delete it if something's wrong. Because of that, the clinic's database should make it easy for authorised staff to update patient info or fix mistakes. Following these laws not only protects patient rights but also helps the clinic stay responsible and trusted.

Ethical Use of Patient Data

Apart from following laws, Totally Toothy needs to make sure it uses patient data ethically too. In healthcare, this is an absolute priority because using medical data the wrong way can actually harm patients and break their trust.

For instance, the system stores allergy information that doctors rely on for safe treatment. If that information got shared without permission or used for something unrelated to healthcare, it would be very unethical. Ethical data use means only using patient info for medical reasons and never sharing it with anyone outside the clinic unless the patient agrees. Handling personal data responsibly is a big part of ethical behaviour in healthcare (Florea and Florea, 2020).

Data Accuracy and Integrity

Another major issue for the Totally Toothy database is keeping data accurate and reliable. Dentists and doctors depend on the database to make correct medical decisions and treatment plans, so if the information isn't accurate, it could cause real problems.

For example, if a patient's allergy is entered wrongly, it might lead to giving them medicine that causes a dangerous reaction. To prevent that, the database should include checks when data is entered and allow quick updates if something changes. Making sure the information is always correct keeps patients safe and helps the clinic give better quality care (Faridoon and Kechadi, 2024).

Professional Responsibility

Finally, the people who work with the Totally Toothy database have a professional duty to handle patient data responsibly. This includes everyone, from doctors and assistants to system administrators. They must all follow confidentiality rules and make sure private information doesn't get shared or exposed. If a data leak happened, it could badly affect the clinic's reputation and make patients lose trust.

People who work in healthcare are also expected to follow ethical guidelines that focus on protecting patients, avoiding harm, and keeping information private. Acting professionally and handling patient data with care helps make sure everything in the Totally Toothy system is run safely, honestly, and ethically (Menlo Report, 2012).

Conclusion

In conclusion, managing and designing the database for Totally Toothy Dental Clinic means more than just storing data, it's also about following professional, legal, and ethical responsibilities. Keeping patient information private, using strong security systems, following data protection laws, and using information ethically are all necessary parts of managing it properly.

Making sure records are always accurate and controlling who can access them plays a big role in protecting patients and keeping their trust. By applying these principles, Totally Toothy Dental Clinic can run a secure, responsible, and trustworthy database system that protects patients' personal information while helping deliver safe and effective healthcare.

Part B – Database Implementation

B1. Relational Model Diagram

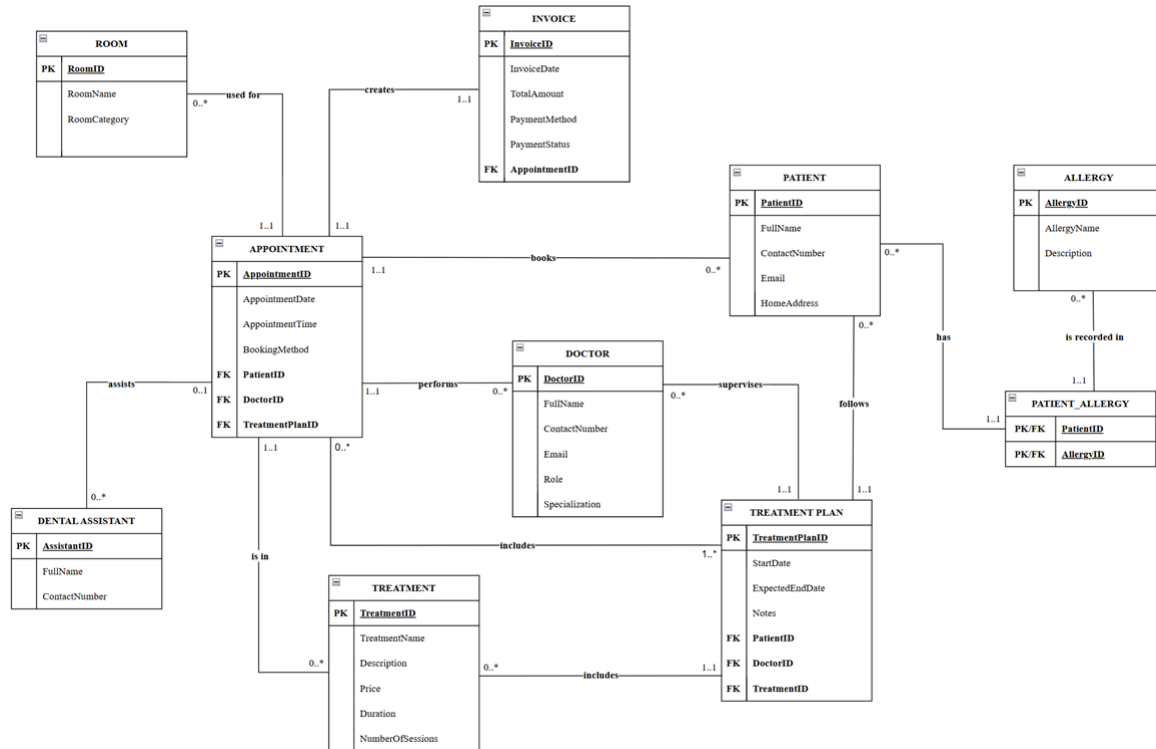


Figure 3 Relational model

B2. Convert Relational Model into Tables

Table Name	Attribute	Data Type	Properties
ROOM	RoomID	INT(5)	PK
	RoomName	VARCHAR(64)	
	RoomCategory	VARCHAR(32)	
INVOICE	InvoiceID	INT(10)	PK
	InvoiceDate	DATE	
	TotalAmount	DECIMAL(10,2)	
	PaymentMethod	VARCHAR(32)	
	PaymentStatus	VARCHAR(32)	

	AppointmentID	INT(10)	FK (APPOINTMENT)
APPOINTMENT	AppointmentID	INT(10)	PK
	AppointmentDate	DATE	
	AppointmentTime	TIME	
	BookingMethod	VARCHAR(32)	
	PatientID	INT(10)	FK (PATIENT)
	DoctorID	INT(10)	FK (DOCTOR)
	TreatmentPlanID	INT(10)	FK (TREATMENT_PLAN)
PATIENT	PatientID	INT(10)	PK
	FullName	VARCHAR(128)	
	ContactNumber	VARCHAR(15)	
	Email	VARCHAR(128)	
	HomeAddress	VARCHAR(255)	
DOCTOR	DoctorID	INT(10)	PK
	FullName	VARCHAR(128)	
	ContactNumber	VARCHAR(15)	
	Email	VARCHAR(128)	
	Role	VARCHAR(32)	
	Specialization	VARCHAR(64)	
TREATMENT	TreatmentID	INT(10)	PK
	TreatmentName	VARCHAR(128)	
	Description	TEXT	
	Price	DECIMAL(10,2)	
	Duration	INT(5)	
	NumberOfSessions	INT(3)	
TREATMENT_PLAN	TreatmentPlanID	INT(10)	PK

	StartDate	DATE	
	ExpectedEndDate	DATE	
	Notes	TEXT	
	PatientID	INT(10)	FK (PATIENT)
	DoctorID	INT(10)	FK (DOCTOR)
	TreatmentID	INT(10)	FK (TREATMENT)
DENTAL_ASSISTANT	AssistantID	INT(10)	PK
	FullName	VARCHAR(128)	
	ContactNumber	VARCHAR(15)	
ALLERGY	AllergyID	INT(10)	PK
	AllergyName	VARCHAR(64)	
	Description	TEXT	
PATIENT_ALLERGY	PatientID	INT(10)	PK, FK (PATIENT)
	AllergyID	INT(10)	PK, FK (ALLERGY)

Justification of Datatypes:**ROOM Table****RoomID - INT(5)**

INT is used because the RoomID is just a number used to identify each room. Numbers are easier for the system to sort and search, so using INT makes the database work faster.

RoomName - VARCHAR(64)

VARCHAR is used because room names are text and can have different lengths. Some room names might be short while others are longer, so VARCHAR lets the system store them without wasting space.

RoomCategory - VARCHAR(32)

This field stores the type of room (like consultation room or treatment room). Since these are words and not numbers, VARCHAR is used to store the text.

INVOICE Table

InvoiceID - INT(10)

INT is used because the invoice ID is a number that uniquely identifies each invoice. Using a number makes it easier for the database to find and manage invoices.

InvoiceDate - DATE

DATE is used because this field stores a calendar date. The DATE datatype keeps the date in the correct format and helps the system sort invoices by date.

TotalAmount - DECIMAL(10,2)

DECIMAL is used because this field stores money. Money needs decimal places (like 100.50), so DECIMAL allows the system to store exact currency values.

PaymentMethod - VARCHAR(32)

This stores how the payment was made (for example cash, card, or insurance). These are words, so VARCHAR is used to store text.

PaymentStatus - VARCHAR(32)

This field shows the status of the payment such as paid, pending, or overdue. Since these values are text, VARCHAR is the most suitable datatype.

AppointmentID - INT(10)

INT is used because this field connects the invoice to the appointment table. The datatype must match the AppointmentID in the appointment table.

APPOINTMENT Table

AppointmentID - INT(10)

INT is used because every appointment needs a unique number so the system can identify it easily.

AppointmentDate - DATE

DATE is used because this field stores the day the appointment will happen.

AppointmentTime - TIME

TIME is used because this field stores the exact time of the appointment, separate from the date.

BookingMethod - VARCHAR(32)

This shows how the appointment was booked, such as online, phone, or walk-in. Since these are words, VARCHAR is used.

PatientID - INT(10)

INT is used because this field links the appointment to a patient. It must be a number because the PatientID in the patient table is also a number.

DoctorID - INT(10)

INT is used because it connects the appointment to the doctor responsible for it.

TreatmentPlanID - INT(10)

This is also a number because it links the appointment to a treatment plan in another table.

PATIENT Table

PatientID - INT(10)

INT is used because each patient needs a unique number so the system can identify them easily.

FullName - VARCHAR(128)

Names are text and can have different lengths, so VARCHAR is used to store them properly.

ContactNumber - VARCHAR(15)

Phone numbers are stored as VARCHAR instead of INT because they can include symbols like “+” and may start with zero.

Email - VARCHAR(128)

Email addresses contain letters and symbols, so they must be stored as text using VARCHAR.

HomeAddress - VARCHAR(255)

Addresses can be quite long and contain different characters, so a larger VARCHAR is used to store the full address.

DOCTOR Table

DoctorID - INT(10)

INT is used because every doctor needs a unique number for identification.

FullName - VARCHAR(128)

Doctor names are text and may vary in length, so VARCHAR is used.

ContactNumber - VARCHAR(15)

Stored as VARCHAR because phone numbers may include different formats and symbols.

Email - VARCHAR(128)

Email addresses are text values, so VARCHAR is used.

Role - VARCHAR(32)

This field stores the doctor's role, such as dentist or specialist. Since these are words, VARCHAR is suitable.

Specialization - VARCHAR(64)

Specialization names are also text, so VARCHAR is used to store them.

TREATMENT Table

TreatmentID - INT(10)

INT is used because each treatment needs a unique number so it can be identified in the system.

TreatmentName - VARCHAR(128)

Treatment names are text and can vary in length, so VARCHAR is used.

Description - TEXT

TEXT is used because the description of a treatment can be long and may not fit in a normal VARCHAR field.

Price - DECIMAL(10,2)

DECIMAL is used because the price is a money value and needs decimal places for accuracy.

Duration - INT(5)

INT is used because the duration is stored as a number (usually minutes).

NumberOfSessions - INT(3)

This stores how many sessions a treatment needs, which is a small number, so INT works well.

TREATMENT_PLAN Table

TreatmentPlanID - INT(10)

INT is used because each treatment plan needs a unique number to identify it.

StartDate - DATE

DATE is used to store when the treatment plan begins.

ExpectedEndDate - DATE

DATE is used to store the expected finishing date of the treatment.

Notes - TEXT

TEXT is used because notes may contain longer explanations or extra details.

PatientID - INT(10)

INT is used because this field links the treatment plan to the patient.

DoctorID - INT(10)

INT is used to connect the treatment plan to the doctor responsible for the treatment.

TreatmentID - INT(10)

INT is used because it links the treatment plan to a specific treatment.

DENTAL_ASSISTANT Table

AssistantID - INT(10)

INT is used because each dental assistant needs a unique ID number.

FullName - VARCHAR(128)

Names are stored as text, so VARCHAR is used.

ContactNumber - VARCHAR(15)

Stored as VARCHAR because phone numbers may have different formats.

ALLERGY Table

AllergyID - INT(10)

INT is used because each allergy needs a unique identifier.

AllergyName - VARCHAR(64)

Allergy names are text values, so VARCHAR is used.

Description - TEXT

TEXT is used because allergy descriptions may be longer.

PATIENT_ALLERGY Table

PatientID - INT(10)

INT is used because it links to the Patient table and shows which patient has the allergy.

AllergyID - INT(10)

INT is used because it links to the Allergy table and helps show the relationship between patients and allergies.

B3. Tables Creation

The database was created using MySQL Workbench. All tables were implemented according to the relational model with appropriate primary and foreign key constraints to maintain referential integrity.

B3.1 CREATE TABLE Statements

The following SQL statements create the database structure:

Create and use the database

CREATE DATABASE TotallyToothyClinic;

USE TotallyToothyClinic;

Create Patient table

CREATE TABLE Patient (

PatientID INT PRIMARY KEY,

FullName VARCHAR(100) NOT NULL,

ContactNumber VARCHAR(15),

Email VARCHAR(100),

HomeAddress VARCHAR(255)

);

Create Doctor table

CREATE TABLE Doctor (

DoctorID INT PRIMARY KEY,

FullName VARCHAR(100) NOT NULL,

ContactNumber VARCHAR(15),


```
Email VARCHAR(100),  
Role VARCHAR(50),  
Specialization VARCHAR(50)  
);
```

#Create Treatment table

```
CREATE TABLE Treatment (  
    TreatmentID INT PRIMARY KEY,  
    TreatmentName VARCHAR(100) NOT NULL,  
    Description VARCHAR(255),  
    Price INT,  
    Duration INT,  
    NumberOfSessions INT  
);
```

Create Room table

```
CREATE TABLE Room (  
    RoomID INT PRIMARY KEY,  
    RoomName VARCHAR(20),  
    RoomCategory VARCHAR(30)  
);
```

Create Allergy table

```
CREATE TABLE Allergy (
```

```
AllergyID INT PRIMARY KEY,  
AllergyName VARCHAR(50) NOT NULL,  
Description VARCHAR(255)  
);
```

#Create Patient_Allergy junction table

```
CREATE TABLE Patient_Allergy (  
    PatientID INT,  
    AllergyID INT,  
    PRIMARY KEY (PatientID, AllergyID),  
    FOREIGN KEY (PatientID) REFERENCES Patient(PatientID),  
    FOREIGN KEY (AllergyID) REFERENCES Allergy(AllergyID)  
);
```

Create TreatmentPlan table

```
CREATE TABLE TreatmentPlan (  
    TreatmentPlanID INT PRIMARY KEY,  
    StartDate DATE NOT NULL,  
    ExpectedEndDate DATE,  
    Notes VARCHAR(255),  
    PatientID INT,  
    DoctorID INT,  
    TreatmentID INT,  
    FOREIGN KEY (PatientID) REFERENCES Patient(PatientID),
```

```
FOREIGN KEY (DoctorID) REFERENCES Doctor(DoctorID),  
FOREIGN KEY (TreatmentID) REFERENCES Treatment(TreatmentID)  
);
```

Create Appointment table with TreatmentPlanID included

```
CREATE TABLE Appointment (  
AppointmentID INT PRIMARY KEY,  
ApptDate DATE NOT NULL,  
ApptTime TIME NOT NULL,  
BookingMethod VARCHAR(20),  
PaymentType VARCHAR(20),  
PatientID INT,  
DoctorID INT,  
TreatmentID INT,  
RoomID INT,  
TreatmentPlanID INT,  
FOREIGN KEY (PatientID) REFERENCES Patient(PatientID),  
FOREIGN KEY (DoctorID) REFERENCES Doctor(DoctorID),  
FOREIGN KEY (TreatmentID) REFERENCES Treatment(TreatmentID),  
FOREIGN KEY (RoomID) REFERENCES Room(RoomID),  
FOREIGN KEY (TreatmentPlanID) REFERENCES TreatmentPlan(TreatmentPlanID)  
);
```

Create Invoice table with 1:1 relationship

```
CREATE TABLE Invoice (  
    InvoiceID INT PRIMARY KEY,  
    Amount INT NOT NULL,  
    IssueDate DATE NOT NULL,  
    PaymentStatus VARCHAR(15),  
    PaymentMethod VARCHAR(20),  
    AppointmentID INT UNIQUE NOT NULL,  
    FOREIGN KEY (AppointmentID) REFERENCES Appointment(AppointmentID)  
);
```

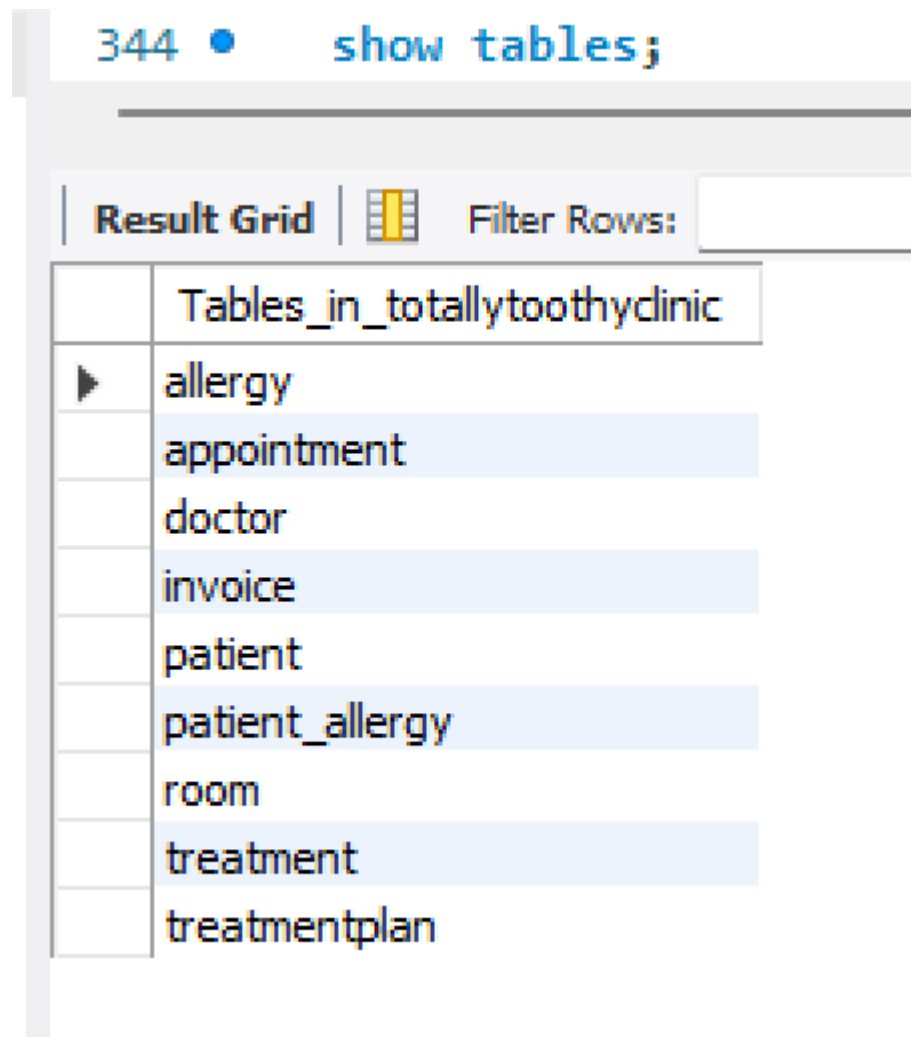


Figure 4 MySQL Workbench showing all nine tables successfully created in the TotallyToothyClinic database

B3.2 Insert Into Table Statements

Sample data was inserted into all tables to test the database integrity and relationships. Each table contains at least 10 records as required:

Insert Doctor data

INSERT INTO Doctor (DoctorID, FullName, ContactNumber, Email, Role, Specialization) VALUES

(1, 'Mutsawashe Mandinde', '0554179651', 'mandindemutsawashe@gmail.com', 'Orthodontist', 'Braces & Alignment'),

(2, 'Reeva Deepak', '0501122334', 'reevadeepak@gmail.com', 'General Dentist', 'Cosmetic Dentistry'),

(3, 'Solomon Orange', '0525556667', 'solomonorange@gmail.com', 'Oral Surgeon', 'Oral Surgery'),

(4, 'Esther Blue', '0589998887', 'estherblue@gmail.com', 'Dental Hygienist', 'Periodontics'),

(5, 'Samuel Sapphire', '0501112222', 'samuelsapphire@toothyclinic.com', 'Pediatric Dentist', 'Childrens Dentistry'),

(6, 'Rachel Rose', '0503334444', 'rachelrose@toothyclinic.com', 'Endodontist', 'Root Canal Specialist'),

(7, 'Gideon Gold', '0505556666', 'gideongold@toothyclinic.com', 'Prosthodontist', 'Dentures & Implants'),

(8, 'Hannah Honey', '0507778888', 'hannahhoney@toothyclinic.com', 'Periodontist', 'Gum Disease Specialist'),

(9, 'Isaac Indigo', '0509990000', 'isaacindigo@toothyclinic.com', 'Oral Surgeon', 'Maxillofacial Surgery'),

(10, 'Phoebe Pear', '0501234567', 'phoebepear@toothyclinic.com', 'General Dentist', 'General Dentistry');

Insert Patient data

*INSERT INTO Patient (PatientID, FullName, ContactNumber, Email, HomeAddress)
VALUES*

(101, 'Isaac Mango', '0567771111', 'isaacmango@gmail.com', 'Villa 42, Jumeirah 1, Dubai'),

(102, 'Gideon Violet', '0567772222', 'gideonviolet@gmail.com', 'Apt 1004, Burj Vista, Downtown Dubai'),

(103, 'Delilah Cherry', '0567773333', 'delilahcherry@gmail.com', 'FronD D, Palm Jumeirah, Dubai'),

(104, 'Ezekiel Lime', '0567774444', 'ezekiellime@gmail.com', '123 Montgomery Drive, Waterfalls, Dubai'),

(105, 'Ruth Silver', '0567775555', 'ruthsilver@gmail.com', 'Apt 220, Marina Gate 2, Dubai Marina'),

(106, 'Noah Pineapple', '0567776666', 'noahpineapple@gmail.com', 'Villa 9, Al Barari, Dubai'),

(107, 'Caleb Cherry', '0567777777', 'calebcherry@gmail.com', 'Villa 5, Arabian Ranches, Dubai'),

(108, 'Lydia Lavender', '0567778888', 'lydialavender@gmail.com', 'Apt 12, City Walk Building 5, Dubai'),

(109, 'Hanna Hazel', '0567779999', 'hannahazel@gmail.com', 'Townhouse 1, Serena, Dubai'),

(110, 'Joshua Jade', '0567770000', 'joshuajade@gmail.com', 'Apt 902, JBR, Dubai'),

(111, 'Sarah Saffron', '0568881111', 'sarahsaffron@gmail.com', 'Villa 33, Al Furjan, Dubai');

Insert Room data

INSERT INTO Room (RoomID, RoomName, RoomCategory) VALUES

(201, 'Surgery 1', 'Orthodontics'),

(202, 'Surgery 2', 'General Dentistry'),

(203, 'X-Ray Suite', 'Diagnostics'),

(204, 'Consultation Room', 'Initial Assessment'),

(205, 'Hygiene Bay', 'Cleaning & Prevention'),

(206, 'Pediatric Wing', 'Kids Dentistry'),

(207, 'Surgery 3', 'Oral Surgery'),

(208, 'Recovery Room', 'Post-Op Care'),

(209, 'Lab 1', 'Prosthetics & Crowns'),

(210, 'Emergency Bay', 'Urgent Care');

Insert Treatment data

```
INSERT INTO Treatment (TreatmentID, TreatmentName, Description, Price, Duration,  
NumberOfSessions) VALUES  
  
(301, 'Routine Cleaning', 'Scaling and polishing for plaque removal', 150, 30, 1),  
(302, 'Invisalign Install', 'Teeth straightening system with invisalign', 5500, 60, 15),  
(303, 'Metal Braces', 'Traditional orthodontic alignment', 3500, 60, 12),  
(304, 'Root Canal', 'Endodontic therapy to save a damaged tooth', 850, 90, 2),  
(305, 'Teeth Whitening', 'Professional laser-assisted whitening', 400, 45, 1),  
(306, 'Dental Implant', 'Titanium post replacement for missing teeth', 8000, 120, 3),  
(307, 'Wisdom Tooth Extraction', 'Surgical removal of impacted wisdom teeth', 1200, 90,  
1),  
(308, 'Porcelain Veneers', 'Cosmetic shells for front teeth smile design', 12000, 180, 2),  
(309, 'Emergency Filling', 'Urgent repair for tooth decay or cavities', 300, 45, 1),  
(310, 'Full Mouth Recon', 'Complete restoration of all teeth in the mouth', 25000, 300, 10);
```

Insert Allergy data

```
INSERT INTO Allergy (AllergyID, AllergyName, Description) VALUES  
  
(401, 'Latex', 'Immediate facial swelling and patient begins looking like a balloon. '),  
(402, 'Lidocaine', 'Severe hallucinations. '),  
(403, 'Amoxicillin', 'Violent pink hives covering 90% of the body. '),  
(404, 'Epinephrine', 'Extreme heart palpitations and patient cannot breathe. '),  
(405, 'Nitrous Oxide', 'Uncontrollable laughing fits followed by weeping. '),  
(406, 'Nickel', 'Rash and swelling if orthodontic wires contain nickel. '),  
(407, 'Chlorhexidine', 'Severe mouth sores from standard surgical mouthwash. '),
```


(408, 'Codeine', 'Nausea, vomiting, and extreme dizziness.')

(409, 'Acrylic', 'Inflammation of gums when in contact with denture bases.')

(410, 'Iodine', 'Anaphylactic risk when using certain antiseptic solutions.');

Insert Patient_Allergy data

INSERT INTO Patient_Allergy (PatientID, AllergyID) VALUES

(101, 403), (102, 402), (104, 401), (106, 405), (103, 408),

(105, 407), (107, 406), (108, 410), (109, 409), (110, 404);

Insert TreatmentPlan data

INSERT INTO TreatmentPlan (TreatmentPlanID, StartDate, ExpectedEndDate, Notes, PatientID, DoctorID, TreatmentID) VALUES

(701, '2026-01-10', '2027-01-10', 'Patient requires monthly tightening.', 101, 1, 303),

(702, '2026-02-15', '2026-02-15', 'Standard maintenance, monitor gum health.', 102, 2, 301),

(703, '2026-02-20', '2026-03-20', 'Multi-stage root canal.', 103, 2, 304),

(704, '2026-03-01', '2026-12-01', 'Invisalign sequence; 15 trays total.', 104, 1, 302),

(705, '2026-03-05', '2026-03-10', 'Two part professional whitening process.', 105, 2, 305),

(706, '2026-01-20', '2026-04-20', 'Follow-up on braces, watch for latex sensitivity.', 106, 1, 303),

(707, '2026-03-10', '2027-03-10', 'Complex alignment for Caleb; consult surgeon if needed.', 107, 1, 303),

(708, '2026-02-01', '2026-02-28', 'Periodontal therapy; 4 sessions total.', 108, 2, 301),

(709, '2026-03-12', '2026-06-12', 'Extensive cosmetic restoration.', 109, 2, 302),

(710, '2026-03-15', '2026-03-15', 'Initial preventative assessment.', 110, 1, 301),

(711, '2026-03-06', '2026-04-06', 'Root canal treatment for tooth #19', 110, 2, 304),

(712, '2026-03-07', '2027-03-07', 'Full braces treatment', 111, 1, 303);

Insert Appointment data with TreatmentPlanID included

INSERT INTO Appointment

*(AppointmentID, ApptDate, ApptTime, BookingMethod, PaymentType, PatientID,
DoctorID, TreatmentID, RoomID, TreatmentPlanID)*

VALUES

(600, '2026-03-10', '09:00:00', 'Online', 'Card', 101, 1, 303, 201, 701),

(610, '2026-03-15', '09:00:00', 'Online', 'Card', 101, 1, 303, 201, 701),

(611, '2026-03-20', '10:00:00', 'Phone', 'Cash', 101, 1, 303, 201, 701),

(601, '2026-03-10', '10:00:00', 'Phone', 'Cash', 102, 2, 301, 205, 702),

(612, '2026-03-16', '11:00:00', 'Online', 'Card', 102, 2, 301, 205, 702),

(613, '2026-03-21', '09:30:00', 'Walk-in', 'Cash', 102, 2, 301, 205, 702),

(602, '2026-03-10', '11:00:00', 'Online', 'Insurance', 103, 3, 307, 207, 703),

(614, '2026-03-17', '09:00:00', 'Online', 'Insurance', 103, 2, 304, 202, 703),

(615, '2026-03-22', '10:30:00', 'Phone', 'Cash', 103, 2, 304, 202, 703),

(616, '2026-03-18', '11:00:00', 'Walk-in', 'Card', 104, 1, 302, 201, 704),

(617, '2026-03-23', '12:00:00', 'Online', 'Card', 104, 1, 302, 201, 704),

(618, '2026-03-24', '09:00:00', 'Phone', 'Cash', 104, 1, 302, 201, 704),

(604, '2026-03-11', '10:30:00', 'Online', 'Card', 105, 5, 305, 206, 705),

(619, '2026-03-19', '10:00:00', 'Online', 'Card', 105, 2, 305, 206, 705),

(620, '2026-03-25', '11:30:00', 'Walk-in', 'Cash', 105, 2, 305, 206, 705),

(605, '2026-03-11', '12:00:00', 'Phone', 'Cash', 106, 6, 304, 202, 706),

(621, '2026-03-19', '12:00:00', 'Online', 'Card', 106, 6, 304, 202, 706),

(622, '2026-03-26', '09:00:00', 'Phone', 'Cash', 106, 6, 304, 202, 706),
(606, '2026-03-12', '09:00:00', 'Online', 'Insurance', 107, 7, 306, 209, 707),
(623, '2026-03-20', '10:30:00', 'Walk-in', 'Card', 107, 7, 306, 209, 707),
(624, '2026-03-27', '11:00:00', 'Online', 'Cash', 107, 7, 306, 209, 707),
(607, '2026-03-12', '10:30:00', 'Walk-in', 'Card', 108, 8, 301, 205, 708),
(625, '2026-03-21', '09:00:00', 'Phone', 'Card', 108, 8, 301, 205, 708),
(626, '2026-03-28', '10:30:00', 'Online', 'Cash', 108, 8, 301, 205, 708),
(608, '2026-03-12', '12:00:00', 'Online', 'Cash', 109, 9, 307, 207, 709),
(627, '2026-03-22', '09:00:00', 'Online', 'Card', 109, 9, 307, 207, 709),
(628, '2026-03-29', '10:30:00', 'Walk-in', 'Cash', 109, 9, 307, 207, 709),
(609, '2026-03-13', '09:00:00', 'Phone', 'Card', 110, 10, 301, 202, 710),
(629, '2026-03-23', '09:00:00', 'Online', 'Card', 110, 10, 301, 202, 710),
(711, '2026-03-06', '09:00:00', 'Phone', 'Cash', 110, 2, 304, 202, 711),
(630, '2026-03-24', '10:00:00', 'Online', 'Card', 111, 1, 303, 201, 712),
(631, '2026-03-25', '11:00:00', 'Phone', 'Cash', 111, 1, 303, 201, 712),
(632, '2026-03-26', '12:00:00', 'Walk-in', 'Card', 111, 1, 303, 201, 712);

Insert Invoice data

INSERT INTO Invoice

(InvoiceID, Amount, IssueDate, PaymentStatus, PaymentMethod, AppointmentID)

VALUES

(950, 3500, '2026-03-10', 'Paid', 'Credit Card', 600),
(963, 3500, '2026-03-15', 'Pending', 'Card', 610),
(964, 3500, '2026-03-20', 'Pending', 'Cash', 611),

(951, 150, '2026-03-10', 'Pending', 'Cash', 601),
(965, 150, '2026-03-16', 'Pending', 'Card', 612),
(966, 150, '2026-03-21', 'Pending', 'Cash', 613),
(952, 1200, '2026-03-10', 'Paid', 'Insurance', 602),
(967, 850, '2026-03-17', 'Pending', 'Insurance', 614),
(968, 850, '2026-03-22', 'Pending', 'Cash', 615),
(969, 5500, '2026-03-18', 'Pending', 'Card', 616),
(970, 5500, '2026-03-23', 'Pending', 'Card', 617),
(971, 5500, '2026-03-24', 'Pending', 'Cash', 618),
(954, 400, '2026-03-11', 'Paid', 'Credit Card', 604),
(972, 400, '2026-03-19', 'Pending', 'Card', 619),
(973, 400, '2026-03-25', 'Pending', 'Cash', 620),
(955, 850, '2026-03-11', 'Pending', 'Cash', 605),
(974, 850, '2026-03-19', 'Pending', 'Card', 621),
(975, 850, '2026-03-26', 'Pending', 'Cash', 622),
(956, 8000, '2026-03-12', 'Paid', 'Insurance', 606),
(976, 8000, '2026-03-20', 'Pending', 'Insurance', 623),
(977, 8000, '2026-03-27', 'Pending', 'Cash', 624),
(957, 150, '2026-03-12', 'Paid', 'Card', 607),
(978, 150, '2026-03-21', 'Pending', 'Card', 625),
(979, 150, '2026-03-28', 'Pending', 'Cash', 626),
(958, 1200, '2026-03-12', 'Paid', 'Cash', 608),
(980, 1200, '2026-03-22', 'Pending', 'Card', 627),
(981, 1200, '2026-03-29', 'Pending', 'Cash', 628),

(959, 150, '2026-03-13', 'Pending', 'Credit Card', 609),

(982, 150, '2026-03-23', 'Pending', 'Card', 629),

(983, 850, '2026-03-06', 'Pending', 'Cash', 711),

(984, 3500, '2026-03-24', 'Pending', 'Card', 630),

(985, 3500, '2026-03-25', 'Pending', 'Cash', 631),

(986, 3500, '2026-03-26', 'Pending', 'Card', 632);

PatientID	FullName	ContactNumber	Email	HomeAddress
101	Isaac Mango	0567771111	isaacmango@gmail.com	Villa 42, Jumeirah 1, Dubai
102	Gideon Violet	0567772222	gideonviolet@gmail.com	Apt 1004, Burj Vista, Downtown Dubai
103	Delilah Cherry	0567773333	delilahcherry@gmail.com	FronD D, Palm Jumeirah, Dubai
104	Ezekiel Lime	0567774444	ezekeillime@gmail.com	123 Montgomery Drive, Waterfalls, Dubai
105	Ruth Silver	0567775555	ruthsilver@gmail.com	Apt 220, Marina Gate 2, Dubai Marina
106	Noah Pineapple	0567776666	noahpineapple@gmail.com	Villa 9, Al Barari, Dubai
107	Caleb Cherry	0567777777	calebcherry@gmail.com	Villa 5, Arabian Ranches, Dubai
108	Lydia Lavender	0567778888	lydialavender@gmail.com	Apt 12, City Walk Building 5, Dubai
109	Hanna Hazel	0567779999	hannahazel@gmail.com	Townhouse 1, Serena, Dubai
110	Joshua Jade	0567770000	joshuajade@gmail.com	Apt 902, JBR, Dubai
111	Sarah Saffron	0568881111	sarahsaffron@gmail.com	Villa 33, Al Furjan, Dubai
NULL	NULL	NULL	NULL	NULL

Figure 5 Patient table showing 11 patient records with personal details and contact information

DoctorID	FullName	ContactNumber	Email	Role	Specialization
1	Mutsawashe Mandinde	0554179651	mandindemutsawashe@gmail.com	Orthodontist	Braces & Alignment
2	Reeva Deepak	0501122334	reevadeepak@gmail.com	General Dentist	Cosmetic Dentistry
3	Solomon Orange	0525556667	solomonorange@gmail.com	Oral Surgeon	Oral Surgery
4	Esther Blue	0589998887	estherblue@gmail.com	Dental Hygienist	Periodontics
5	Samuel Sapphire	0501112222	samuelsapphire@toothyclinic.com	Pediatric Dentist	Childrens Dentistry
6	Rachel Rose	0503334444	rachelrose@toothyclinic.com	Endodontist	Root Canal Specialist
7	Gideon Gold	0505556666	gideongold@toothyclinic.com	Prosthodontist	Dentures & Implants
8	Hannah Honey	0507778888	hannahhoney@toothyclinic.com	Periodontist	Gum Disease Specialist
9	Isaac Indigo	0509990000	isaacindigo@toothyclinic.com	Oral Surgeon	Maxillofacial Surgery
10	Phoebe Pear	0501234567	phoebepear@toothyclinic.com	General Dentist	General Dentistry
NULL	NULL	NULL	NULL	NULL	NULL

Figure 6 Doctor table displaying 10 dental professionals with their roles and specializations

Result Grid						
Filter Rows:		Edit: Export/Import: Wrap Cell Content:				
TreatmentID	TreatmentName	Description	Price	Duration	NumberOfSessions	
301	Routine Cleaning	Scaling and polishing for plaque removal	150	30	1	
302	Invisalign Install	Teeth straightening system with invisalign	5500	60	15	
303	Metal Braces	Traditional orthodontic alignment	3500	60	12	
304	Root Canal	Endodontic therapy to save a damaged tooth	850	90	2	
305	Teeth Whitening	Professional laser-assisted whitening	400	45	1	
306	Dental Implant	Titanium post replacement for missing teeth	8000	120	3	
307	Wisdom Tooth Extraction	Surgical removal of impacted wisdom teeth	1200	90	1	
308	Porcelain Veneers	Cosmetic shells for front teeth smile design	12000	180	2	
309	Emergency Filling	Urgent repair for tooth decay or cavities	300	45	1	
310	Full Mouth Recon	Complete restoration of all teeth in the mouth	25000	300	10	
NULL	NULL	NULL	NULL	NULL	NULL	

Figure 7 Treatment table listing all dental procedures offered with prices and duration

Result Grid			
Filter Rows:		Edit:	
RoomID	RoomName	RoomCategory	
201	Surgery 1	Orthodontics	
202	Surgery 2	General Dentistry	
203	X-Ray Suite	Diagnostics	
204	Consultation Room	Initial Assessment	
205	Hygiene Bay	Cleaning & Prevention	
206	Pediatric Wing	Kids Dentistry	
207	Surgery 3	Oral Surgery	
208	Recovery Room	Post-Op Care	
209	Lab 1	Prosthetics & Crowns	
210	Emergency Bay	Urgent Care	
NULL	NULL	NULL	

Figure 8 Room table showing clinic facilities and room categories

Result Grid			
	Filter Rows:		Edit: Export/
	AllergyID	AllergyName	Description
▶	401	Latex	Immediate facial swelling and patient begins loo...
	402	Lidocaine	Severe hallucinations.
	403	Amoxicillin	Violent pink hives covering 90% of the body.
	404	Epinephrine	Extreme heart palpitations and patient cannot b...
	405	Nitrous Oxide	Uncontrollable laughing fits followed by weeping.
	406	Nickel	Rash and swelling if orthodontic wires contain ni...
	407	Chlorhexidine	Severe mouth sores from standard surgical mou...
	408	Codeine	Nausea, vomiting, and extreme dizziness.
	409	Acrylic	Inflammation of gums when in contact with dent...
	410	Iodine	Anaphylactic risk when using certain antiseptic s...
✱	NULL	NULL	NULL

Figure 9 Allergy table containing allergy names and descriptions for patient medical records

Result Grid		
	Filter	
	PatientID	AllergyID
▶	104	401
	102	402
	101	403
	110	404
	106	405
	107	406
	105	407
	103	408
	109	409
	108	410
✱	NULL	NULL

Figure 10 Allergy table containing allergy names and descriptions for patient medical records

Result Grid							
Filter Rows:							
Edit:							
Export/Import:							
Wrap Cell Content:							
TreatmentPlanID	StartDate	ExpectedEndDate	Notes	PatientID	DoctorID	TreatmentID	
701	2026-01-10	2027-01-10	Patient requires monthly tightening.	101	1	303	
702	2026-02-15	2026-02-15	Standard maintenance, monitor gum health.	102	2	301	
703	2026-02-20	2026-03-20	Multi-stage root canal.	103	2	304	
704	2026-03-01	2026-12-01	Invisalign sequence; 15 trays total.	104	1	302	
705	2026-03-05	2026-03-10	Two part professional whitening process.	105	2	305	
706	2026-01-20	2026-04-20	Follow-up on braces, watch for latex sensitivity.	106	1	303	
707	2026-03-10	2027-03-10	Complex alignment for Caleb; consult surgeon if...	107	1	303	
708	2026-02-01	2026-02-28	Periodontal therapy; 4 sessions total.	108	2	301	
709	2026-03-12	2026-06-12	Extensive cosmetic restoration.	109	2	302	
710	2026-03-15	2026-03-15	Initial preventative assessment.	110	1	301	
711	2026-03-06	2026-04-06	Root canal treatment for tooth #19	110	2	304	
712	2026-03-07	2027-03-07	Full braces treatment	111	1	303	
NULL	NULL	NULL	NULL	NULL	NULL	NULL	

Figure 11 TreatmentPlan table showing long-term treatment plans with start dates, end dates and notes

Result Grid										
Filter Rows:										
Edit:										
Export/Import:										
Wrap Cell Content:										
AppointmentID	ApptDate	ApptTime	BookingMethod	PaymentType	PatientID	DoctorID	TreatmentID	RoomID	TreatmentPlanID	
600	2026-03-10	09:00:00	Online	Card	101	1	303	201	701	
601	2026-03-10	10:00:00	Phone	Cash	102	2	301	205	702	
602	2026-03-10	11:00:00	Online	Insurance	103	3	307	207	703	
604	2026-03-11	10:30:00	Online	Card	105	5	305	206	705	
605	2026-03-11	12:00:00	Phone	Cash	106	6	304	202	706	
606	2026-03-12	09:00:00	Online	Insurance	107	7	306	209	707	
607	2026-03-12	10:30:00	Walk-in	Card	108	8	301	205	708	
608	2026-03-12	12:00:00	Online	Cash	109	9	307	207	709	
609	2026-03-13	09:00:00	Phone	Card	110	10	301	202	710	
610	2026-03-15	09:00:00	Online	Card	101	1	303	201	701	
611	2026-03-20	10:00:00	Phone	Cash	101	1	303	201	701	
612	2026-03-16	11:00:00	Online	Card	102	2	301	205	702	
613	2026-03-21	09:30:00	Walk-in	Cash	102	2	301	205	702	
614	2026-03-17	09:00:00	Online	Insurance	103	2	304	202	703	
615	2026-03-22	10:30:00	Phone	Cash	103	2	304	202	703	
616	2026-03-18	11:00:00	Walk-in	Card	104	1	302	201	704	
617	2026-03-23	12:00:00	Online	Card	104	1	302	201	704	
618	2026-03-24	09:00:00	Phone	Cash	104	1	302	201	704	
619	2026-03-19	10:00:00	Online	Card	105	2	305	206	705	
620	2026-03-25	11:30:00	Walk-in	Cash	105	2	305	206	705	
621	2026-03-19	12:00:00	Online	Card	106	6	304	202	706	
622	2026-03-26	09:00:00	Phone	Cash	106	6	304	202	706	
623	2026-03-20	10:30:00	Walk-in	Card	107	7	306	209	707	
624	2026-03-27	11:00:00	Online	Cash	107	7	306	209	707	
625	2026-03-21	09:00:00	Phone	Card	108	8	301	205	708	
626	2026-03-28	10:30:00	Online	Cash	108	8	301	205	708	
627	2026-03-22	09:00:00	Online	Card	109	9	307	207	709	
628	2026-03-29	10:30:00	Walk-in	Cash	109	9	307	207	709	
629	2026-03-23	09:00:00	Online	Card	110	10	301	202	710	
630	2026-03-24	10:00:00	Online	Card	111	1	303	201	712	
631	2026-03-25	11:00:00	Phone	Cash	111	1	303	201	712	

Figure 12 Appointment table displaying scheduled appointments with dates, times and assigned staff

CST2340- DATABASE SYSTEMS: DESIGN AND IMPLICATION

Result Grid						
		Filter Rows:			Edit:	Export/Import:
	InvoiceID	Amount	IssueDate	PaymentStatus	PaymentMethod	AppointmentID
▶	950	3500	2026-03-10	Paid	Credit Card	600
	951	150	2026-03-10	Pending	Cash	601
	952	1200	2026-03-10	Paid	Insurance	602
	954	400	2026-03-11	Paid	Credit Card	604
	955	850	2026-03-11	Pending	Cash	605
	956	8000	2026-03-12	Paid	Insurance	606
	957	150	2026-03-12	Paid	Card	607
	958	1200	2026-03-12	Paid	Cash	608
	959	150	2026-03-13	Pending	Credit Card	609
	963	3500	2026-03-15	Pending	Card	610
	964	3500	2026-03-20	Pending	Cash	611
	965	150	2026-03-16	Pending	Card	612
	966	150	2026-03-21	Pending	Cash	613
	967	850	2026-03-17	Pending	Insurance	614
	968	850	2026-03-22	Pending	Cash	615
	969	5500	2026-03-18	Pending	Card	616
	970	5500	2026-03-23	Pending	Card	617
	971	5500	2026-03-24	Pending	Cash	618
	972	400	2026-03-19	Pending	Card	619
	973	400	2026-03-25	Pending	Cash	620
	974	850	2026-03-19	Pending	Card	621
	975	850	2026-03-26	Pending	Cash	622
	976	8000	2026-03-20	Pending	Insurance	623
	977	8000	2026-03-27	Pending	Cash	624
	978	150	2026-03-21	Pending	Card	625
	979	150	2026-03-28	Pending	Cash	626
	980	1200	2026-03-22	Pending	Card	627
	981	1200	2026-03-29	Pending	Cash	628
	982	150	2026-03-23	Pending	Card	629
	983	850	2026-03-06	Pending	Cash	711
	984	3500	2026-03-24	Pending	Card	630
▶	985	3500	2026-03-25	Pending	Card	631
▶	986	3500	2026-03-26	Pending	Card	632
▶	987	3500	2026-03-27	Pending	Card	633
▶	988	3500	2026-03-28	Pending	Card	634
▶	989	3500	2026-03-29	Pending	Card	635
▶	990	3500	2026-03-30	Pending	Card	636
▶	991	3500	2026-03-31	Pending	Card	637
▶	992	3500	2026-04-01	Pending	Card	638
▶	993	3500	2026-04-02	Pending	Card	639
▶	994	3500	2026-04-03	Pending	Card	640
▶	995	3500	2026-04-04	Pending	Card	641
▶	996	3500	2026-04-05	Pending	Card	642
▶	997	3500	2026-04-06	Pending	Card	643
▶	998	3500	2026-04-07	Pending	Card	644
▶	999	3500	2026-04-08	Pending	Card	645
▶	1000	3500	2026-04-09	Pending	Card	646
▶	1001	3500	2026-04-10	Pending	Card	647
▶	1002	3500	2026-04-11	Pending	Card	648
▶	1003	3500	2026-04-12	Pending	Card	649
▶	1004	3500	2026-04-13	Pending	Card	650
▶	1005	3500	2026-04-14	Pending	Card	651
▶	1006	3500	2026-04-15	Pending	Card	652
▶	1007	3500	2026-04-16	Pending	Card	653
▶	1008	3500	2026-04-17	Pending	Card	654
▶	1009	3500	2026-04-18	Pending	Card	655
▶	1010	3500	2026-04-19	Pending	Card	656
▶	1011	3500	2026-04-20	Pending	Card	657
▶	1012	3500	2026-04-21	Pending	Card	658
▶	1013	3500	2026-04-22	Pending	Card	659
▶	1014	3500	2026-04-23	Pending	Card	660
▶	1015	3500	2026-04-24	Pending	Card	661
▶	1016	3500	2026-04-25	Pending	Card	662
▶	1017	3500	2026-04-26	Pending	Card	663
▶	1018	3500	2026-04-27	Pending	Card	664
▶	1019	3500	2026-04-28	Pending	Card	665
▶	1020	3500	2026-04-29	Pending	Card	666
▶	1021	3500	2026-04-30	Pending	Card	667
▶	1022	3500	2026-05-01	Pending	Card	668
▶	1023	3500	2026-05-02	Pending	Card	669
▶	1024	3500	2026-05-03	Pending	Card	670
▶	1025	3500	2026-05-04	Pending	Card	671
▶	1026	3500	2026-05-05	Pending	Card	672
▶	1027	3500	2026-05-06	Pending	Card	673
▶	1028	3500	2026-05-07	Pending	Card	674
▶	1029	3500	2026-05-08	Pending	Card	675
▶	1030	3500	2026-05-09	Pending	Card	676
▶	1031	3500	2026-05-10	Pending	Card	677
▶	1032	3500	2026-05-11	Pending	Card	678
▶	1033	3500	2026-05-12	Pending	Card	679
▶	1034	3500	2026-05-13	Pending	Card	680
▶	1035	3500	2026-05-14	Pending	Card	681
▶	1036	3500	2026-05-15	Pending	Card	682
▶	1037	3500	2026-05-16	Pending	Card	683
▶	1038	3500	2026-05-17	Pending	Card	684
▶	1039	3500	2026-05-18	Pending	Card	685
▶	1040	3500	2026-05-19	Pending	Card	686
▶	1041	3500	2026-05-20	Pending	Card	687
▶	1042	3500	2026-05-21	Pending	Card	688
▶	1043	3500	2026-05-22	Pending	Card	689
▶	1044	3500	2026-05-23	Pending	Card	690
▶	1045	3500	2026-05-24	Pending	Card	691
▶	1046	3500	2026-05-25	Pending	Card	692
▶	1047	3500	2026-05-26	Pending	Card	693
▶	1048	3500	2026-05-27	Pending	Card	694
▶	1049	3500	2026-05-28	Pending	Card	695
▶	1050	3500	2026-05-29	Pending	Card	696
▶	1051	3500	2026-05-30	Pending	Card	697
▶	1052	3500	2026-05-31	Pending	Card	698
▶	1053	3500	2026-06-01	Pending	Card	699
▶	1054	3500	2026-06-02	Pending	Card	700
▶	1055	3500	2026-06-03	Pending	Card	701
▶	1056	3500	2026-06-04	Pending	Card	702
▶	1057	3500	2026-06-05	Pending	Card	703
▶	1058	3500	2026-06-06	Pending	Card	704
▶	1059	3500	2026-06-07	Pending	Card	705
▶	1060	3500	2026-06-08	Pending	Card	706
▶	1061	3500	2026-06-09	Pending	Card	707
▶	1062	3500	2026-06-10	Pending	Card	708
▶	1063	3500	2026-06-11	Pending	Card	709
▶	1064	3500	2026-06-12	Pending	Card	710
▶	1065	3500	2026-06-13	Pending	Card	711
▶	1066	3500	2026-06-14	Pending	Card	712
▶	1067	3500	2026-06-15	Pending	Card	713
▶	1068	3500	2026-06-16	Pending	Card	714
▶	1069	3500	2026-06-17	Pending	Card	715
▶	1070	3500	2026-06-18	Pending	Card	716
▶	1071	3500	2026-06-19	Pending	Card	717
▶	1072	3500	2026-06-20	Pending	Card	718
▶	1073	3500	2026-06-21	Pending	Card	719
▶	1074	3500	2026-06-22	Pending	Card	720
▶	1075	3500	2026-06-23	Pending	Card	721
▶	1076	3500	2026-06-24	Pending	Card	722
▶	1077	3500	2026-06-25	Pending	Card	723
▶	1078	3500	2026-06-26	Pending	Card	724
▶	1079	3500	2026-06-27	Pending	Card	725
▶	1080	3500	2026-06-28	Pending	Card	726
▶	1081	3500	2026-06-29	Pending	Card	727
▶	1082	3500	2026-06-30	Pending	Card	728
▶	1083	3500	2026-07-01	Pending	Card	729
▶	1084	3500	2026-07-02	Pending	Card	730
▶	1085	3500	2026-07-03	Pending	Card	731
▶	1086	3500	2026-07-04	Pending	Card	732
▶	1087	3500	2026-07-05	Pending	Card	733
▶	1088	3500	2026-07-06	Pending	Card	734
▶	1089	3500	2026-07-07	Pending	Card	735
▶	1090	3500	2026-07-08	Pending	Card	736
▶	1091	3500	2026-07-09	Pending	Card	737
▶	1092	3500	2026-07-10	Pending	Card	738
▶	1093	3500	2026-07-11	Pending	Card	739
▶	1094	3500	2026-07-12	Pending	Card	740
▶	1095	3500	2026-07-13	Pending	Card	741
▶	1096	3500	2026-07-14	Pending	Card	742
▶	1097	3500	2026-07-15	Pending	Card	743
▶	1098	3500	2026-07-16	Pending	Card	744
▶	1099	3500	2026-07-17	Pending	Card	745
▶	1100	3500	2026-07-18	Pending	Card	746
▶	1101	3500	2026-07-19	Pending	Card	747
▶	1102	3500	2026-07-20	Pending	Card	748
▶	1103	3500	2026-07-21	Pending	Card	749
▶	1104	3500	2026-07-22	Pending	Card	750
▶	1105	3500	2026-07-23	Pending	Card	751
▶	1106	3500	2026-07-24	Pending	Card	752
▶	1107	3500	2026-07-25	Pending	Card	753
▶	1108	3500	2026-07-26	Pending	Card	754
▶	1109	3500	2026-07-27	Pending	Card	755
▶	1110	3500	2026-07-28	Pending	Card	756
▶	1111	3500	2026-07-29	Pending	Card	757
▶	1112	3500	2026-07-30	Pending	Card	758
▶	1113	3500	2026-07-31	Pending	Card	759
▶	1114	3500	2026-08-01	Pending	Card	760
▶	1115	3500	2026-08-02	Pending	Card	761
▶	1116	3500	2026-08-03	Pending	Card	762
▶	1117	3500	2026-08-04	Pending	Card	763
▶	1118	3500	2026-08-05	Pending	Card	764
▶	1119	3500	2026-08-06	Pending	Card	765
▶	1120	3500	2026-08-07	Pending	Card	766
▶	1121	3500	2026-08-08	Pending	Card	767
▶	1122	3500	2026-08-09	Pending	Card	768
▶	1123	3500	2026-08-10	Pending	Card	769
▶	1124	3500	2026-08-11	Pending	Card	770
▶	1125	3500	2026-08-12	Pending	Card	771
▶	1126	3500	2026-08-13	Pending	Card	772
▶	1127	3500	2026-08-14	Pending	Card	773
▶	1128	3500	2026-08-15	Pending	Card	774
▶	1129	3500	2026-08-16	Pending	Card	775
▶	1130	3500	2026-08-17	Pending	Card	776
▶	1131	3500	2026-08-18	Pending	Card	777
▶	1132	3500	2026-08-19	Pending	Card	778
▶	1133	3500	2026-08-20	Pending	Card	779
▶	1134	3500	2026-08-21	Pending	Card	780
▶	1135	3500	2026-08-22	Pending	Card	781
▶	1136	3500	2026-08-23	Pending	Card	782
▶	1137	3500	2026-08-24	Pending	Card	783
▶	1138	3500	2026-08-25	Pending	Card	784
▶	1139	3500	2026-08-26	Pending	Card	785
▶	1140	3500	2026-08-27	Pending	Card	786
▶	1141	3500	2026-08-28	Pending	Card	787
▶	1142	3500				

Part C – SQL Codes

The following queries demonstrate various SQL techniques including joins, subqueries, aggregation, and set operations. Each query includes the SQL code, explanation, and output screenshot.

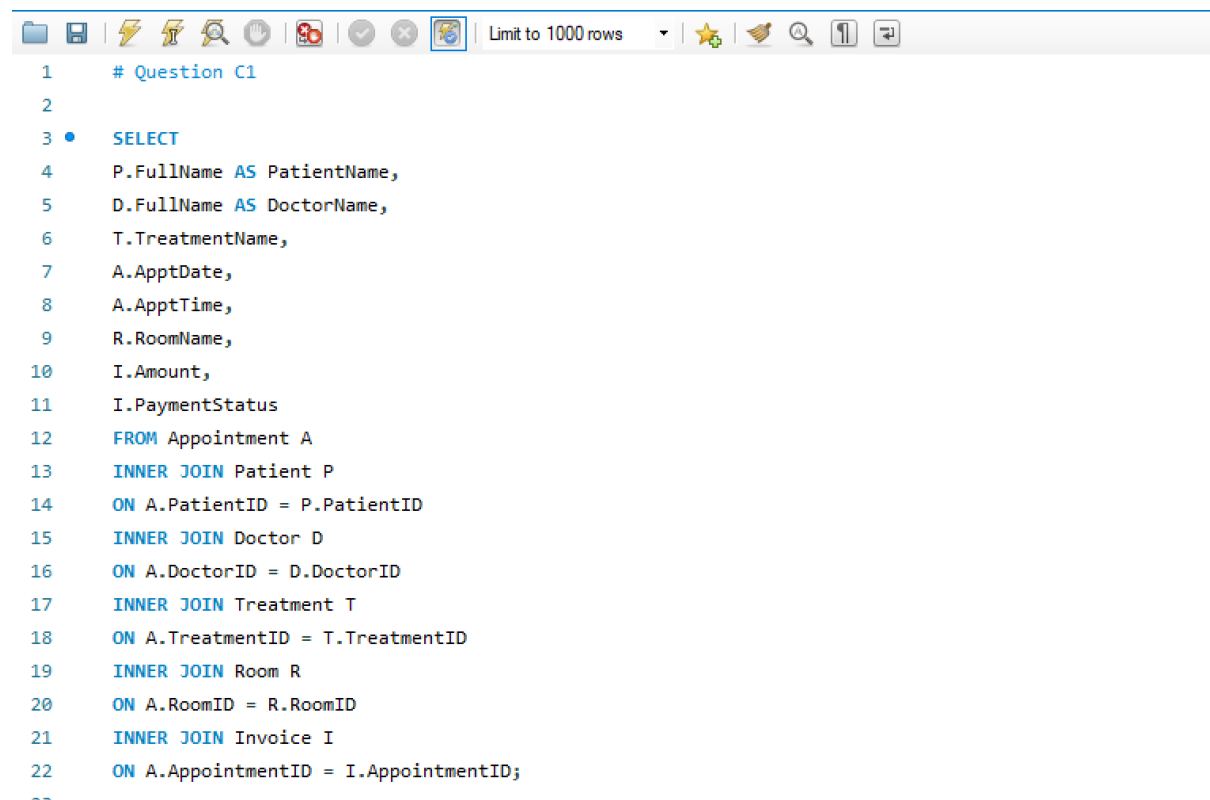
Question C1 - Appointment and Billing Report

- Join of multiple tables to create a complete clinic report

English Sentence:

List the appointments with the patient's full name, doctor's name, treatment received, appointment date and time, room used, invoice amount, and payment status, by joining the Appointment, Treatment, Doctor, Patient, Room, and Billing tables to create a complete clinic report.

SQL Query:



```
1 # Question C1
2
3 • SELECT
4 P.FullName AS PatientName,
5 D.FullName AS DoctorName,
6 T.TreatmentName,
7 A.ApptDate,
8 A.ApptTime,
9 R.RoomName,
10 I.Amount,
11 I.PaymentStatus
12 FROM Appointment A
13 INNER JOIN Patient P
14 ON A.PatientID = P.PatientID
15 INNER JOIN Doctor D
16 ON A.DoctorID = D.DoctorID
17 INNER JOIN Treatment T
18 ON A.TreatmentID = T.TreatmentID
19 INNER JOIN Room R
20 ON A.RoomID = R.RoomID
21 INNER JOIN Invoice I
22 ON A.AppointmentID = I.AppointmentID;
23 ~
```

Explanation:

This query combines the Appointment, Treatment, Doctor, Patient, Room, and Billing tables to create a full clinic report. It shows each patient's name, doctor, treatment, appointment date and time, room, invoice amount, and payment status. By bringing together operational and financial data, the report gives a complete picture of both medical and business activities.

This helps the clinic keep a track of service delivery, payments, and identify patterns in activity. Joining the tables gives a complete and accurate overview of patient care and billing, making management easier and clearer.

Output:

Result Grid Filter Rows: Export: Wrap Cell Content:								
PatientName	DoctorName	TreatmentName	ApptDate	ApptTime	RoomName	Amount	PaymentStatus	
Ruth Silver	Reeva Deepak	Teeth Whitening	2026-03-19	10:00:00	Pediatric Wing	400	Pending	
Ruth Silver	Reeva Deepak	Teeth Whitening	2026-03-25	11:30:00	Pediatric Wing	400	Pending	
Noah Pineapple	Rachel Rose	Root Canal	2026-03-11	12:00:00	Surgery 2	850	Pending	
Noah Pineapple	Rachel Rose	Root Canal	2026-03-19	12:00:00	Surgery 2	850	Pending	
Noah Pineapple	Rachel Rose	Root Canal	2026-03-26	09:00:00	Surgery 2	850	Pending	
Caleb Cherry	Gideon Gold	Dental Implant	2026-03-12	09:00:00	Lab 1	8000	Paid	
Caleb Cherry	Gideon Gold	Dental Implant	2026-03-20	10:30:00	Lab 1	8000	Pending	
Caleb Cherry	Gideon Gold	Dental Implant	2026-03-27	11:00:00	Lab 1	8000	Pending	
Lydia Lavender	Hannah Honey	Routine Cleaning	2026-03-12	10:30:00	Hygiene Bay	150	Paid	
Lydia Lavender	Hannah Honey	Routine Cleaning	2026-03-21	09:00:00	Hygiene Bay	150	Pending	
Lydia Lavender	Hannah Honey	Routine Cleaning	2026-03-28	10:30:00	Hygiene Bay	150	Pending	
Hanna Hazel	Isaac Indigo	Wisdom Tooth Extraction	2026-03-12	12:00:00	Surgery 3	1200	Paid	
Hanna Hazel	Isaac Indigo	Wisdom Tooth Extraction	2026-03-22	09:00:00	Surgery 3	1200	Pending	
Hanna Hazel	Isaac Indigo	Wisdom Tooth Extraction	2026-03-29	10:30:00	Surgery 3	1200	Pending	
Joshua Jade	Phoebe Pear	Routine Cleaning	2026-03-13	09:00:00	Surgery 2	150	Pending	
Joshua Jade	Phoebe Pear	Routine Cleaning	2026-03-23	09:00:00	Surgery 2	150	Pending	
Joshua Jade	Reeva Deepak	Root Canal	2026-03-06	09:00:00	Surgery 2	850	Pending	
Sarah Saffron	Mutsawashe Mandinde	Metal Braces	2026-03-24	10:00:00	Surgery 1	3500	Pending	
Sarah Saffron	Mutsawashe Mandinde	Metal Braces	2026-03-25	11:00:00	Surgery 1	3500	Pending	
Sarah Saffron	Mutsawashe Mandinde	Metal Braces	2026-03-26	12:00:00	Surgery 1	3500	Pending	

Question C2 - Area-Based Appointment Overview

Filtered view of appointments by residential area to support targeted marketing

English Sentence:

List the appointments showing patient name, doctor name, treatment name and appointment date, for patients who live in Dubai Marina.

SQL Query:

```
1 • use totallytoothyclinic;
2   # Find all appointments for patients who live in Dubai Marina
3 •   SELECT
4       p.FullName AS PatientName,
5       d.FullName AS DoctorName,
6       t.TreatmentName,
7       a.ApptDate
8   FROM Appointment a
9   JOIN Patient p ON a.PatientID = p.PatientID
10  JOIN Doctor d ON a.DoctorID = d.DoctorID
11  JOIN Treatment t ON a.TreatmentID = t.TreatmentID
12  WHERE p.HomeAddress LIKE '%Dubai Marina%'
13  ORDER BY a.ApptDate;
```

Explanation:

This query joins the Appointment, Patient, Doctor and Treatment tables to create a complete view of all appointments. The INNER JOIN connects each appointment to its corresponding patient, doctor and treatment details using their respective foreign keys (PatientID, DoctorID, TreatmentID). The WHERE clause filters the results to only include patients whose home address contains 'Dubai Marina' using the LIKE operator with wildcards. This allows the clinic to isolate appointment activity for a specific residential area. The clinic can use this information to target local marketing campaigns or send appointment reminders to patients in this area. The results show that Ruth Silver, a Dubai Marina resident, has had three teeth whitening appointments with two different doctors.

Output:

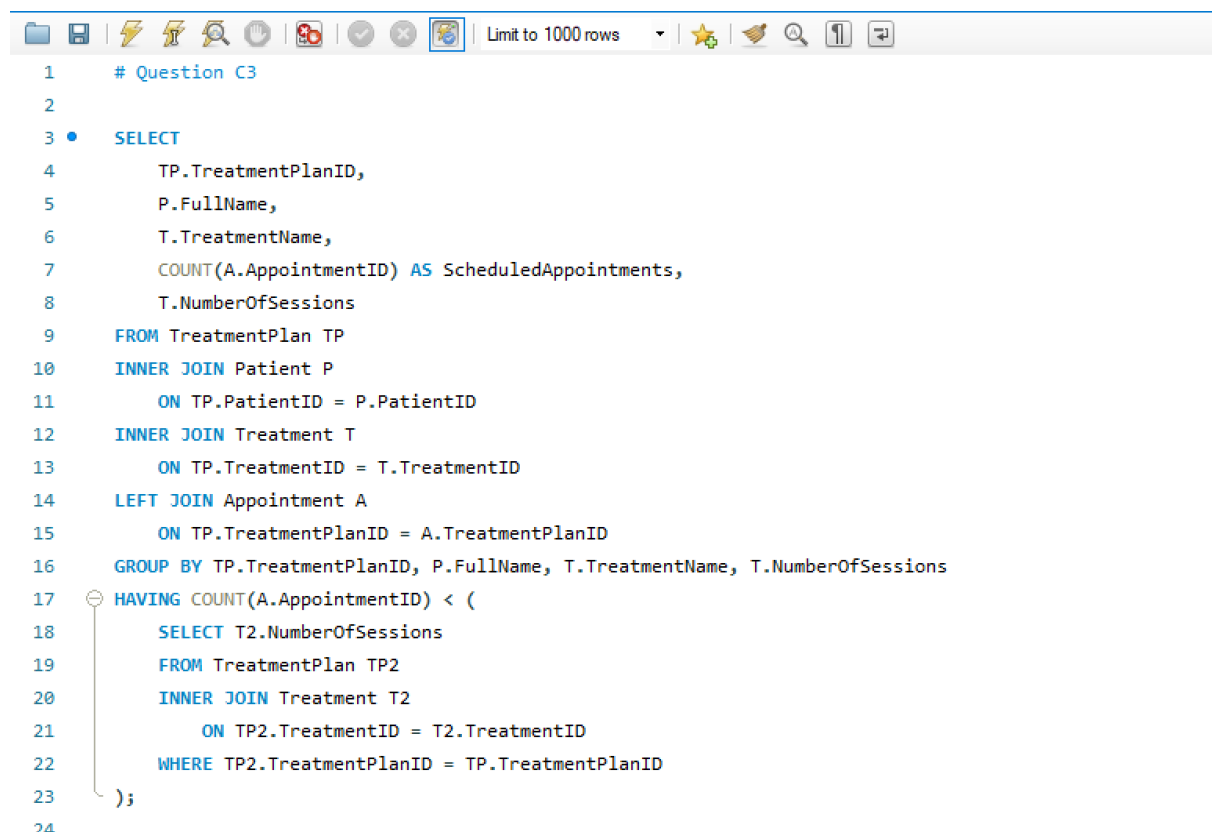
Result Grid				
 Filter Rows:				
Export:  Wra				
	PatientName	DoctorName	TreatmentName	ApptDate
▶	Ruth Silver	Samuel Sapphire	Teeth Whitening	2026-03-11
	Ruth Silver	Reeva Deepak	Teeth Whitening	2026-03-19
	Ruth Silver	Reeva Deepak	Teeth Whitening	2026-03-25

Question C3 - Treatment Plans with Low Appointment Numbers

- Subquery comparing appointments and required sessions

English Sentence:

List the treatment plans with treatment_plan_id, patient name, treatment name, number of booked appointments, and required sessions, for patients whose scheduled appointments are fewer than the sessions required for their treatment plans.

SQL Query:

```
1  # Question C3
2
3  •  SELECT
4      TP.TreatmentPlanID,
5      P.FullName,
6      T.TreatmentName,
7      COUNT(A.AppointmentID) AS ScheduledAppointments,
8      T.NumberOfSessions
9  FROM TreatmentPlan TP
10 INNER JOIN Patient P
11     ON TP.PatientID = P.PatientID
12 INNER JOIN Treatment T
13     ON TP.TreatmentID = T.TreatmentID
14 LEFT JOIN Appointment A
15     ON TP.TreatmentPlanID = A.TreatmentPlanID
16 GROUP BY TP.TreatmentPlanID, P.FullName, T.TreatmentName, T.NumberOfSessions
17 HAVING COUNT(A.AppointmentID) < (
18     SELECT T2.NumberOfSessions
19     FROM TreatmentPlan TP2
20     INNER JOIN Treatment T2
21         ON TP2.TreatmentID = T2.TreatmentID
22     WHERE TP2.TreatmentPlanID = TP.TreatmentPlanID
23 );
24
```

Explanation:

This query checks whether patients have booked all the sessions their treatment plans require. It compares the number of scheduled appointments with the total sessions needed, using a subquery that counts appointments associated to each treatment plan. The output includes treatment plan ID, patient name, treatment name, booked sessions, and required sessions. Staff can quickly see which patients are behind on schedule and need extra appointments. This helps ensure treatments are completed and makes managing doctors' schedules easier by finding gaps in current ongoing treatment plans.

Output:

Result Grid   Filter Rows: Export:  Wrap Cell Content: 					
	TreatmentPlanID	FullName	TreatmentName	ScheduledAppointments	NumberOfSessions
▶	701	Isaac Mango	Metal Braces	3	12
	704	Ezekiel Lime	Invisalign Install	3	15
	706	Noah Pineapple	Metal Braces	3	12
	707	Caleb Cherry	Metal Braces	3	12
	709	Hanna Hazel	Invisalign Install	3	15
	711	Joshua Jade	Root Canal	0	2
	712	Sarah Saffron	Metal Braces	3	12

Question C4 - Same Doctor Patient List

Identifies different patients who visit the same doctor for scheduling insights

English Sentence:

List pairs of different patients who are treated by the same doctor, showing both patient names and the doctor they share.

SQL Query:

```
# Find pairs of patients who see the same doctor
SELECT DISTINCT
    p1.FullName AS Patient1,
    p2.FullName AS Patient2,
    d.FullName AS DoctorName
FROM Appointment a1
JOIN Appointment a2 ON a1.DoctorID = a2.DoctorID AND a1.PatientID < a2.PatientID
JOIN Patient p1 ON a1.PatientID = p1.PatientID
JOIN Patient p2 ON a2.PatientID = p2.PatientID
JOIN Doctor d ON a1.DoctorID = d.DoctorID
ORDER BY d.FullName, p1.FullName;
```

Explanation:

This query uses a self-join on the Appointment table to find pairs of different patients who are treated by the same doctor. By joining the Appointment table to itself on DoctorID and ensuring the patient IDs are different, the query compares each appointment with every other appointment that shares the same doctor. This shows all possible patient combinations per doctor. The clinic can use this to understand patient distribution among staff and manage appointment scheduling more efficiently. The results show that Dr. Mandinde treats Isaac Mango, Ezekiel Lime and Sarah Saffron, while Dr. Deepak treats Gideon Violet, Delilah Cherry, Ruth Silver and Joshua Jade.

	Patient1	Patient2	DoctorName
▶	Ezekiel Lime	Sarah Saffron	Mutsawashe Mandinde
	Isaac Mango	Ezekiel Lime	Mutsawashe Mandinde
	Isaac Mango	Sarah Saffron	Mutsawashe Mandinde
	Delilah Cherry	Ruth Silver	Reeva Deepak
	Delilah Cherry	Joshua Jade	Reeva Deepak
	Gideon Violet	Delilah Cherry	Reeva Deepak
	Gideon Violet	Ruth Silver	Reeva Deepak
	Gideon Violet	Joshua Jade	Reeva Deepak
	Ruth Silver	Joshua Jade	Reeva Deepak

Question C5 - Treatment Demand and Revenue

- Aggregation of treatment data to analyse performance

English Sentence:

List the treatments with treatment name, total number of times performed, and total revenue generated from invoices, for treatments performed more than once and generating more than 3000 in revenue.

SQL Query:

```

1  # Question C5
2
3  •  SELECT
4      T.TreatmentName,
5      COUNT(A.AppointmentID) AS TimesPerformed,
6      SUM(I.Amount) AS TotalRevenue
7  FROM Treatment T
8  INNER JOIN Appointment A
9      ON T.TreatmentID = A.TreatmentID
10 INNER JOIN Invoice I
11     ON A.AppointmentID = I.AppointmentID
12 GROUP BY T.TreatmentName
13 HAVING COUNT(A.AppointmentID) > 1
14     AND SUM(I.Amount) > 3000;
--

```

Explanation:

This query finds which treatments are the most popular and generate the most revenue. It starts with the Treatment table and joins Appointment and Invoice tables. Using COUNT, it

calculates how many times each treatment was done, and using SUM, it totals the revenue from invoices. The GROUP BY makes sure that the calculations are done per treatment, and the HAVING clause filters treatments done more than once and earning over 3000. This helps the management see treatments most in demand, make pricing and staffing decisions, and connect daily clinical activities with financial results.

Output:

Result Grid			Filter Rows:	Export: 	Wrap Cell Content: 
	TreatmentName	TimesPerformed	TotalRevenue		
▶	Metal Braces	6	21000		
	Root Canal	6	5100		
	Dental Implant	3	24000		
	Wisdom Tooth Extraction	4	4800		

Question C6 – UNION

Provides a complete contact list of everyone associated with the clinic for easy reference

English Sentence:

List all patients and doctors associated with the clinic showing their full names, email addresses and a column indicating whether they are a patient or doctor.

SQL Query:

```
#List all people associated with the clinic including patients and doctors
SELECT FullName, Email, 'Patient' AS Type
FROM Patient
UNION
SELECT FullName, Email, 'Doctor' AS Type
FROM Doctor
ORDER BY FullName;
```

Explanation:

This query uses the UNION operator to combine results from two separate SELECT statements into a single list. The first SELECT retrieves all patient names and emails from the Patient table and adds a 'Patient' label under a derived Type column. The second SELECT does the same for doctors from the Doctor table with a 'Doctor' label. Because UNION removes duplicate rows by default, each person appears only once even if they somehow appeared in both tables. The ORDER BY clause sorts the final list alphabetically by name. This creates a complete contact directory by combining patient and doctor records into a single list. The clinic can use this for mass communications such as newsletters, appointment reminders or clinic announcements. The results show all 11 patients and 10 doctors associated with the clinic, sorted alphabetically.

Output:

Result Grid   Filter Rows: Export:  Wrap Cell			
	FullName	Email	Type
▶	Caleb Cherry	calebcherry@gmail.com	Patient
	Delilah Cherry	delilahcherry@gmail.com	Patient
	Esther Blue	estherblue@gmail.com	Doctor
	Ezekiel Lime	ezekeillime@gmail.com	Patient
	Gideon Gold	gideongold@toothyclinic.com	Doctor
	Gideon Violet	gideonviolet@gmail.com	Patient
	Hanna Hazel	hannahazel@gmail.com	Patient
	Hannah Honey	hannahhoney@toothyclinic.com	Doctor
	Isaac Indigo	isaacindigo@toothyclinic.com	Doctor
	Isaac Mango	isaacmango@gmail.com	Patient
	Joshua Jade	joshuajade@gmail.com	Patient
	Lydia Lavender	lydialavender@gmail.com	Patient
	Mutsawashe ...	mandindemutsawashe@gmail.c...	Doctor
	Noah Pineapple	noahpineapple@gmail.com	Patient
	Phoebe Pear	phoebepear@toothyclinic.com	Doctor
	Rachel Rose	rachelrose@toothyclinic.com	Doctor
	Reeva Deepak	reevadeepak@gmail.com	Doctor
	Ruth Silver	ruthsilver@gmail.com	Patient
	Samuel Sapphire	samuelsapphire@toothyclinic.com	Doctor
	Sarah Saffron	sarahsaffron@gmail.com	Patient
	Solomon Orange	solomonorange@gmail.com	Doctor

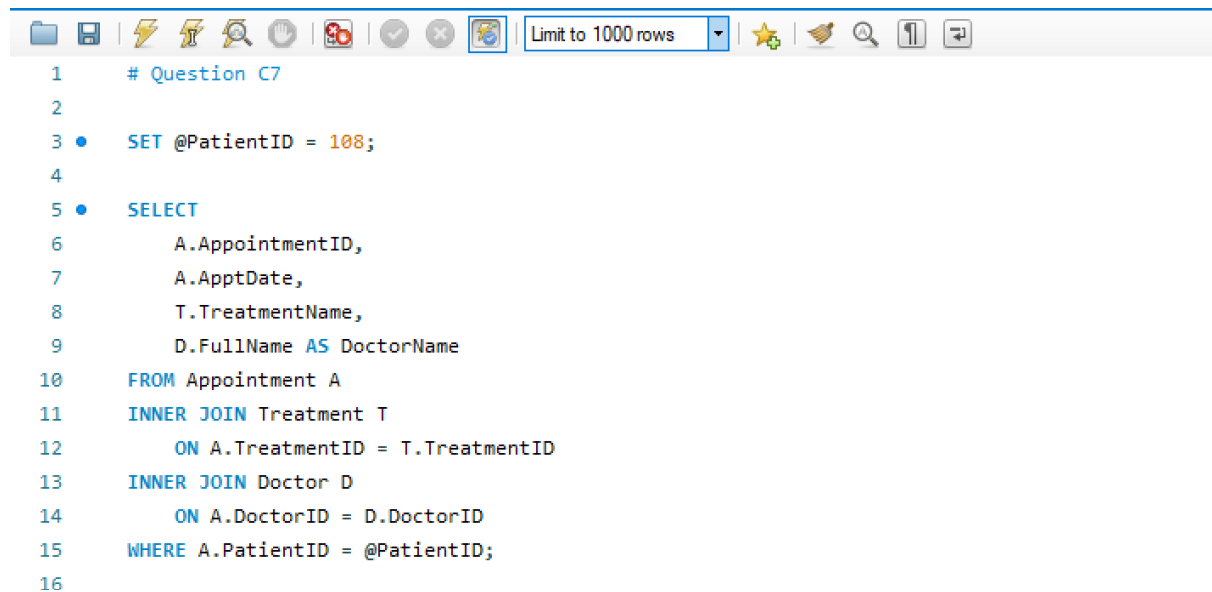
Question C7 - Patient Appointment Lookup

- Interactive query that lets users retrieve patient records

English Sentence:

List the appointments with appointment_id, appointment date, treatment name, and doctor name, for a patient identified by the patient_id entered by the user.

SQL Query:

A screenshot of a SQL query editor window. The window has a toolbar at the top with icons for file operations, execution, and search. A dropdown menu shows "Limit to 1000 rows". The query text is as follows:

```
1  # Question C7
2
3  • SET @PatientID = 108;
4
5  • SELECT
6      A.AppointmentID,
7      A.ApptDate,
8      T.TreatmentName,
9      D.FullName AS DoctorName
10 FROM Appointment A
11 INNER JOIN Treatment T
12     ON A.TreatmentID = T.TreatmentID
13 INNER JOIN Doctor D
14     ON A.DoctorID = D.DoctorID
15 WHERE A.PatientID = @PatientID;
16
```

Explanation:

This query lets receptionists quickly look up a patient's appointments by entering their ID (using a parameter like @PatientID). It filters the Appointment table and joins Treatment and Doctor tables, so that each appointment shows the associated treatment and doctor involved. The result table shows a patient's full appointment history (appointment id, dates, treatments, and doctors) so that staff can confirm schedules or check treatment progress fast. This improves efficiency, makes it unnecessary to search through all the records separately, and helps provide overall better services during patient interactions.

Output:

Result Grid   Filter Rows:					Export: 	Wrap Cell Content: 
	AppointmentID	ApptDate	TreatmentName	DoctorName		
▶	607	2026-03-12	Routine Cleaning	Hannah Honey		
	625	2026-03-21	Routine Cleaning	Hannah Honey		
	626	2026-03-28	Routine Cleaning	Hannah Honey		

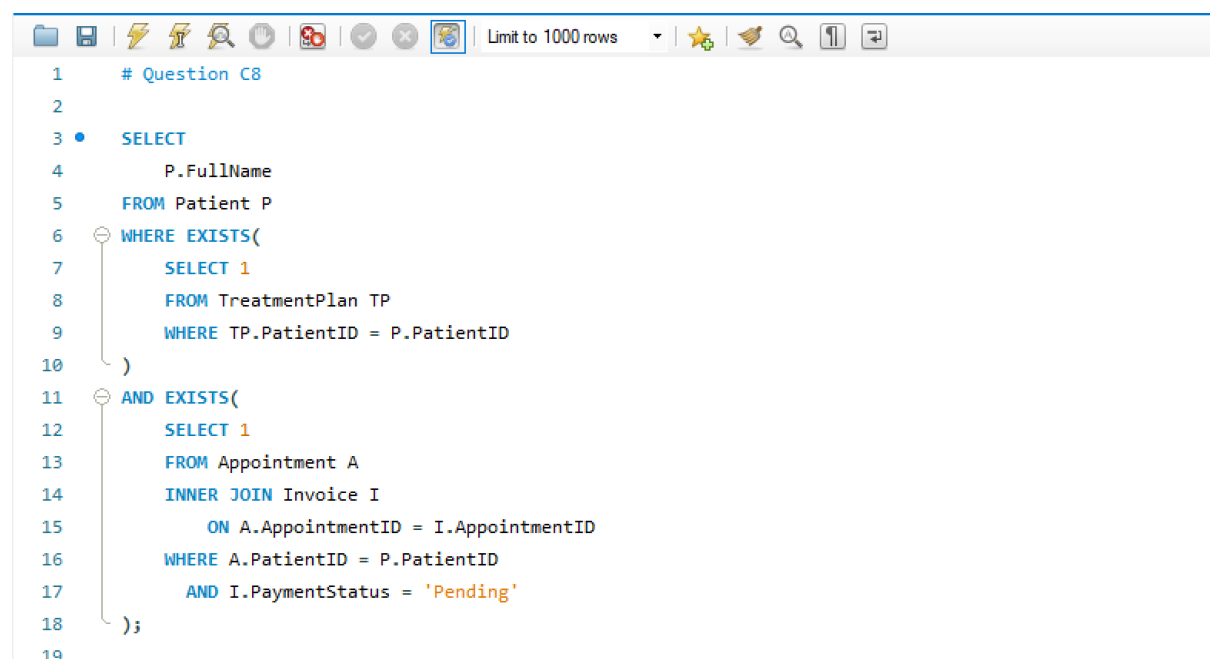
Question C8 - Patients with Treatment Plans and Pending Invoices

- Query using EXISTS to check financial and clinical conditions

English Sentence:

List the patients with patient name who have treatment plans and at least one unpaid invoice linked to their appointments.

SQL Query:

The image shows a screenshot of a SQL query editor window. The title bar includes icons for file operations and a dropdown menu set to "Limit to 1000 rows". The query text is as follows:

```
1  # Question C8
2
3  •  SELECT
4      P.FullName
5  FROM Patient P
6  WHERE EXISTS(
7      SELECT 1
8      FROM TreatmentPlan TP
9      WHERE TP.PatientID = P.PatientID
10 )
11 AND EXISTS(
12     SELECT 1
13     FROM Appointment A
14     INNER JOIN Invoice I
15         ON A.AppointmentID = I.AppointmentID
16     WHERE A.PatientID = P.PatientID
17         AND I.PaymentStatus = 'Pending'
18 );
19
```

Explanation:

This query finds patients who currently have active treatment plans and unpaid invoices. It uses two EXISTS, one to confirm the patient has a treatment plan, and another one to check if there's at least one pending invoice linked through the Appointment and Invoice tables. Only patients falling under both conditions will appear. This helps the clinic identify ongoing treatment cases that need payment follow ups without stopping treatment. Using EXISTS makes the query efficient and helps the clinic manage billing while consistently keeping patient treatment and care.

Output:

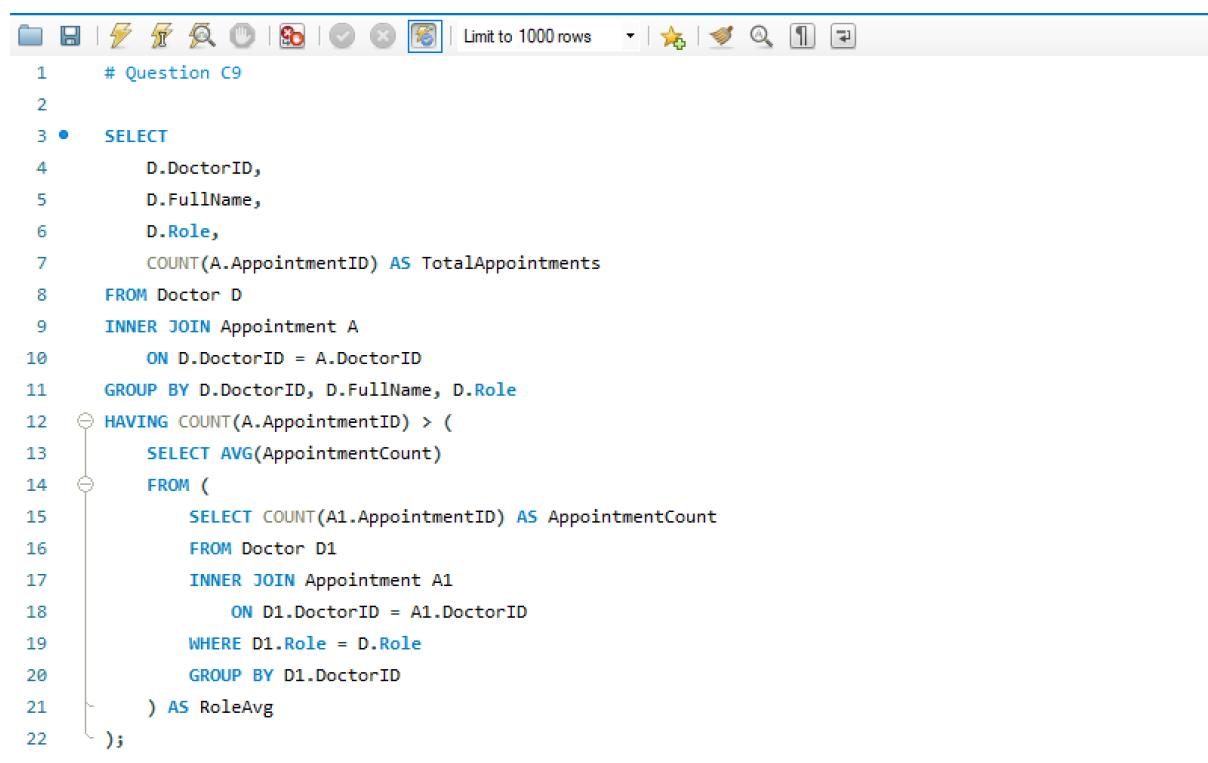
Result Grid				Filter Rows:	Export: 	Wrap Cell Content: 
	FullName					
▶	Joshua Jade					
	Gideon Violet					
	Noah Pineapple					
	Isaac Mango					
	Delilah Cherry					
	Ezekiel Lime					
	Ruth Silver					
	Caleb Cherry					
	Lydia Lavender					
	Hanna Hazel					
	Sarah Saffron					

Question C9 - Doctors with High Workload

- Correlated subquery for role-average comparison

English Sentence:

List the doctors with doctor_id, doctor name, role, and total number of appointments handled, for doctors whose appointment count is higher than the average number of appointments for doctors with the same role.

SQL Query:


```

1  # Question C9
2
3  •  SELECT
4      D.DoctorID,
5      D.FullName,
6      D.Role,
7      COUNT(A.AppointmentID) AS TotalAppointments
8  FROM Doctor D
9  INNER JOIN Appointment A
10     ON D.DoctorID = A.DoctorID
11  GROUP BY D.DoctorID, D.FullName, D.Role
12  HAVING COUNT(A.AppointmentID) > (
13      SELECT AVG(AppointmentCount)
14      FROM (
15          SELECT COUNT(A1.AppointmentID) AS AppointmentCount
16          FROM Doctor D1
17          INNER JOIN Appointment A1
18             ON D1.DoctorID = A1.DoctorID
19          WHERE D1.Role = D.Role
20          GROUP BY D1.DoctorID
21      ) AS RoleAvg
22  );

```

Explanation:

This query finds doctors who have more appointments than the average number handled by others in the same role. It connects the Doctor and Appointment tables to count each doctor's total appointments. The GROUP BY clause groups and organizes the results by doctor, while the HAVING clause keeps only those doctors, whose appointment count is above their role average, calculated using a subquery. This helps the management identify doctors with heavier workloads compared to their coworkers.

Output:

CST2340- DATABASE SYSTEMS: DESIGN AND IMPLICATION

Result Grid				
 Filter Rows:				
Export: 				
Wrap Cell Content: 				
	DoctorID	FullName	Role	TotalAppointments
▶	2	Reeva Deepak	General Dentist	8
	9	Isaac Indigo	Oral Surgeon	3

Question C10 - Complex Treatment Patients

Identifies patients who have undergone multiple major procedures

English Sentence:

List the patients who have undergone both root canal and wisdom tooth extraction, showing their patient ID and full name.

```
326      # Find patients who have had ALL treatments in a specific set(Root Canal and Wisdom Tooth Extraction)
327 •    SELECT
328         p.PatientID,
329         p.FullName AS PatientName
330     FROM Patient p
331     WHERE NOT EXISTS (
332         SELECT t.TreatmentID
333         FROM Treatment t
334         WHERE t.TreatmentID IN (304, 307) #Root Canal and Wisdom Tooth Extraction
335         AND NOT EXISTS (
336             SELECT *
337             FROM Appointment a
338             WHERE a.PatientID = p.PatientID
339             AND a.TreatmentID = t.TreatmentID
340         )
341     );
```

SQL Query:

This query uses the relational division pattern with double NOT EXISTS to find patients who have undergone both specified treatments. The outer NOT EXISTS checks if there is any treatment in the set (Root Canal and Wisdom Tooth Extraction) that the patient has NOT had. The inner NOT EXISTS checks each treatment against the patient's appointment history. If for a given patient, there is no treatment in the set that they have missed, then they have had all of them. This double negative logic is the standard way to implement relational division in SQL. The clinic can use this to identify patients who have had multiple major procedures for follow-up care, payment plan options, or to monitor for potential complications. The results show that Delilah Cherry is the only patient who has had both root canal and wisdom tooth extraction.

Output:

Result Grid			  Filter Rows:	
	PatientID	PatientName		
▶	103	Delilah Cherry		
●	NULL	NULL		

CONTRIBUTION TABLE:

PART	QUESTION	CONTRIBUTION
PART A	A1	- Reeva Josephine Deepak: M01036971 - Mutsawashe Audrey Mandinde: M01033270
	A2	- Reeva Josephine Deepak: M01036971 - Mutsawashe Audrey Mandinde: M01033270
	A3	- Reeva Josephine Deepak: M01036971
	A4	- Mutsawashe Audrey Mandinde: M01033270
	A5	- Reeva Josephine Deepak: M01036971
PART B	B1	- Reeva Josephine Deepak: M01036971 - Mutsawashe Audrey Mandinde: M01033270

	B2	- Reeva Josephine Deepak: M01036971
	B3	- Mutsawashe Audrey Mandinde: M01033270
PART C	C1	- Reeva Josephine Deepak: M01036971
	C2	- Mutsawashe Audrey Mandinde: M01033270
	C3	- Reeva Josephine Deepak: M01036971
	C4	- Mutsawashe Audrey Mandinde: M01033270
	C5	- Reeva Josephine Deepak: M01036971
	C6	- Mutsawashe Audrey Mandinde: M01033270
	C7	- Reeva Josephine Deepak: M01036971

	C8	- Reeva Josephine Deepak: M01036971
	C9	- Reeva Josephine Deepak: M01036971
	C10	- Mutsawashe Audrey Mandinde: M01033270

REFERENCES LIST:

- Faridoon, A. and Tahar, K.M. (2024) Healthcare Data Governance, Privacy, and Security - A Conceptual Framework. Available at: <https://arxiv.org/abs/2403.17648>.
- Florea, D. and Florea, S. (2020) ‘Big Data and the Ethical Implications of Data Privacy in Higher Education Research’, Sustainability, 12(20), p.8744. Available at: <https://doi.org/10.3390/su12208744>.
- International Society of Nephrology (2025) Legal, ethical, privacy, and security issues. Available at: <https://www.theisn.org/in-action/research/share-rr/legal-ethical-privacy-and-security-issues/>.
- Lee, W., Zankl, W. and Chang, H. (2016) ‘An Ethical Approach to Data Privacy Protection’, ISACA Journal. Available at: <https://www.isaca.org/resources/isaca-journal/issues/2016/volume-6/an-ethical-approach-to-data-privacy-protection>.
- Pina, E., Ramos, J., Jorge, H., Váz, P., Silva, J., Wanzeller, C., Abbasi, M. and Martins, P. (2024) ‘Data Privacy and Ethical Considerations in Database Management’, Journal of Cybersecurity and Privacy, 4(3), pp.494–517. Available at: <https://doi.org/10.3390/jcp4030024>.
- The Computing Teacher (2025) Ethical and Legal Issues. Available at: <https://www.thecomputingteacher.com/computer-science/managing-data/ethicallegal?> (Accessed: 12 March 2026).
- Wikipedia Contributors (2019) Privacy by Design. Available at: https://en.wikipedia.org/wiki/Privacy_by_Design.
- Wikipedia (2022) Menlo Report. Available at: https://en.wikipedia.org/wiki/Menlo_Report
- IBM (2022) *NoSQL Databases*, *Ibm.com*. Available at: <https://www.ibm.com/think/topics/nosql-databases>.